06 树

- 二叉树的计数
- 线索化二叉树
- 堆
- 霍夫曼树

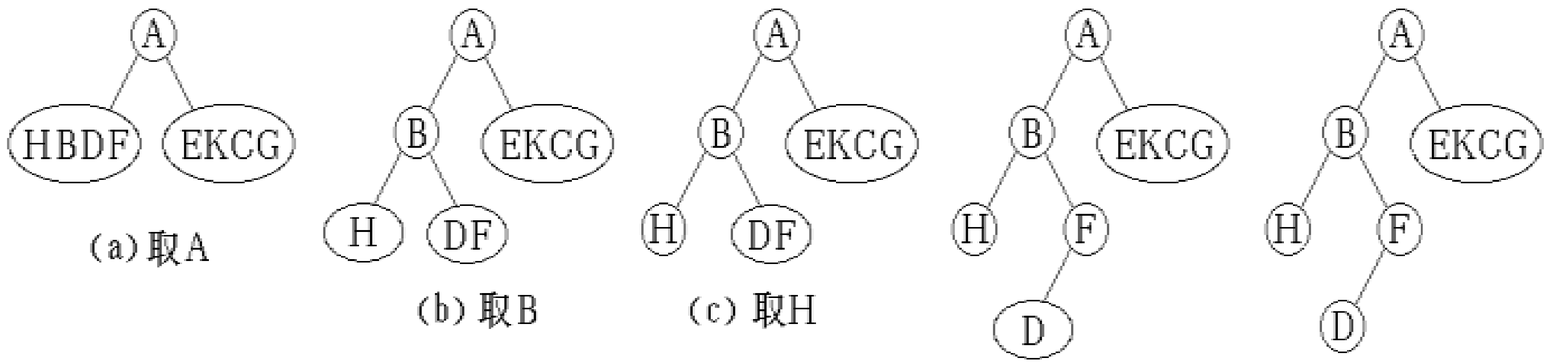
06 树

- 二叉树的计数
- 线索化二叉树
- 堆
- 霍夫曼树

利用前序序列和中序序列构造二叉树

- 由二叉树的前序序列和中序序列可唯一确定一棵二叉树

前序 {ABHFDECKG}，中序 {HBDFAEKCG}



利用前序序列和中序序列构造二叉树

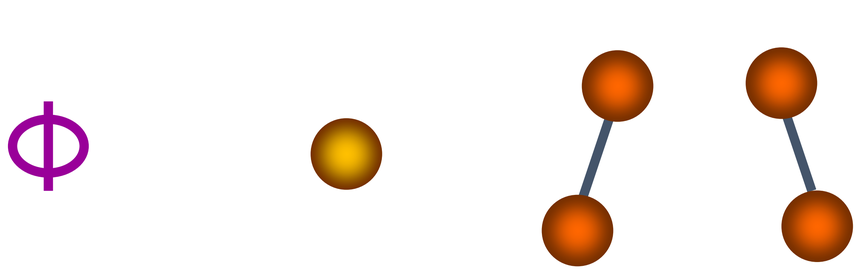
```
template <class Type> BinaryTree <Type> * BinaryTree <Type> ::
CreateBinaryTree(Type *VLR, Type *LVR, int n) {
    if (n==0)  return NULL;
    int k = 0;
    while (VLR[0] != LVR[k])  k++;  // 在中序序列中寻找根
    // 创建根结点
    BinTreeNode <Type> * t = new BinTreeNode <Type> (VLR[0]);
    // 从前序VLR+1开始对中序的0~k-1左子序列
    // 的k个元素递归建立左子树
    t->leftChild = CreateBinaryTree(VLR+1, LVR, k);
    // 从前序VLR+k+1开始对中序的k+1~n-1右子序列
    // 的n-k-1个元素递归建立右子树
    t->rightChild = CreateBinaryTree(VLR+k+1, LVR+k+1, n-k-1);
    return t;
}
```

二叉树的计数

- 具有n个结点的不同二叉树有b_n种
- 对二叉树的结点进行编号，使得其前序序列是1, 2, …, n, 则b_n等于具有前序序列为1, 2, …, n的二叉树可能得到的不同中序序列的数目 定-议二
  - 前序序列是二叉树递归遍历中的进栈顺序
  - 中序序列是二叉树递归遍历中的出栈顺序
  - b_n等于进栈序列为1, 2, …, n的不同出栈序列的数目
- 设1是第i+1个出栈的元素，则1前面的i个元素有b_i种出栈序列，而1后面的n-i-1个元素b_{n-i-1}种出栈序列，因此有

$$b_n = \sum_{i=0}^{n-1} b_i \cdot b_{n-i-1}$$

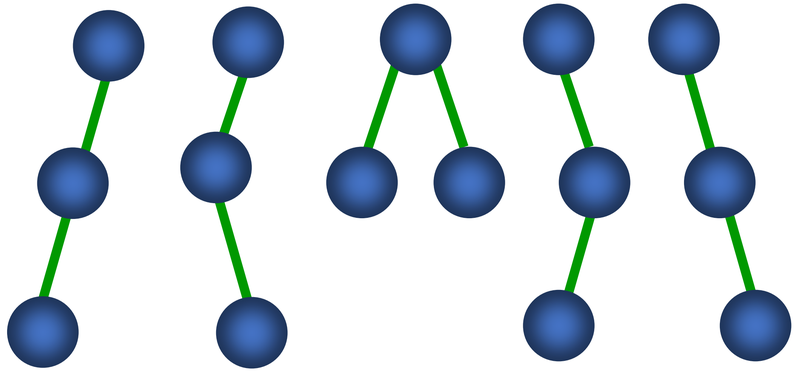
有0, 1, 2, 3, 4个结点的不同二叉树



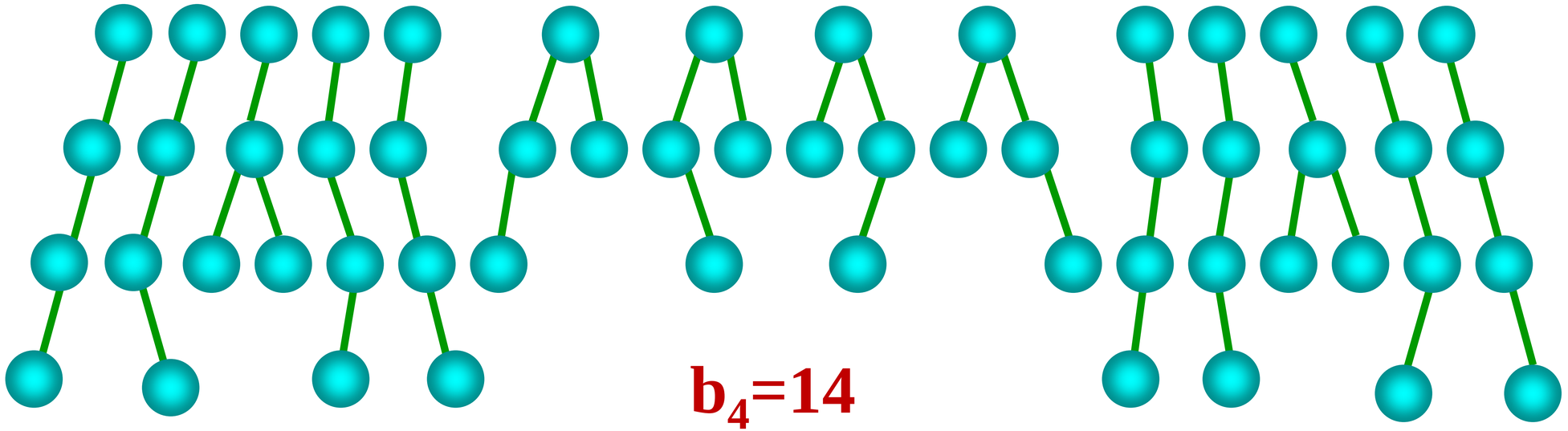
$b_0=1$

$b_1=1$

$b_2=2$



$b_3=5$



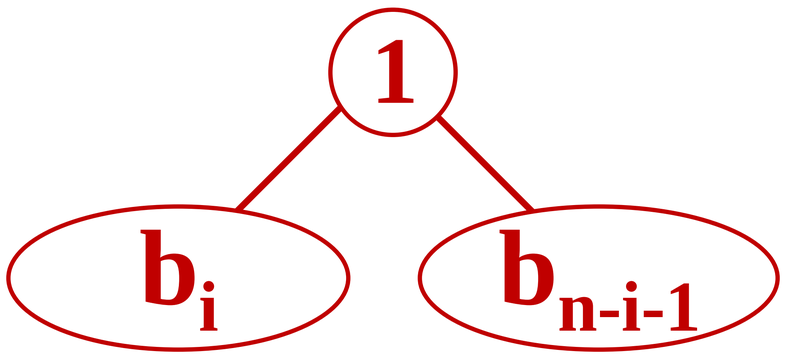
$b_4=14$

二叉树的计数

- b_i表示二叉树中由i个结点的左子树个数
- b_{n-i-1}表示二叉树中由n-i-1个结点的右子树个数
- $b_i * b_{n-i-1}$表示由根结点、有i个结点左子树和有n-i-1个结点的右子树组成的二叉树个数
- 不同二叉树个数等于左子树上可能的不同二叉树个数与右子树上可能的不同二叉树个数的乘积

$$b_n = \sum_{i=0}^{n-1} b_i \cdot b_{n-i-1}$$

$$b_n = \frac{1}{n+1} C_{2n}^{n} = \frac{1}{n+1} \frac{(2n)!}{n! \cdot n!}$$



06 树

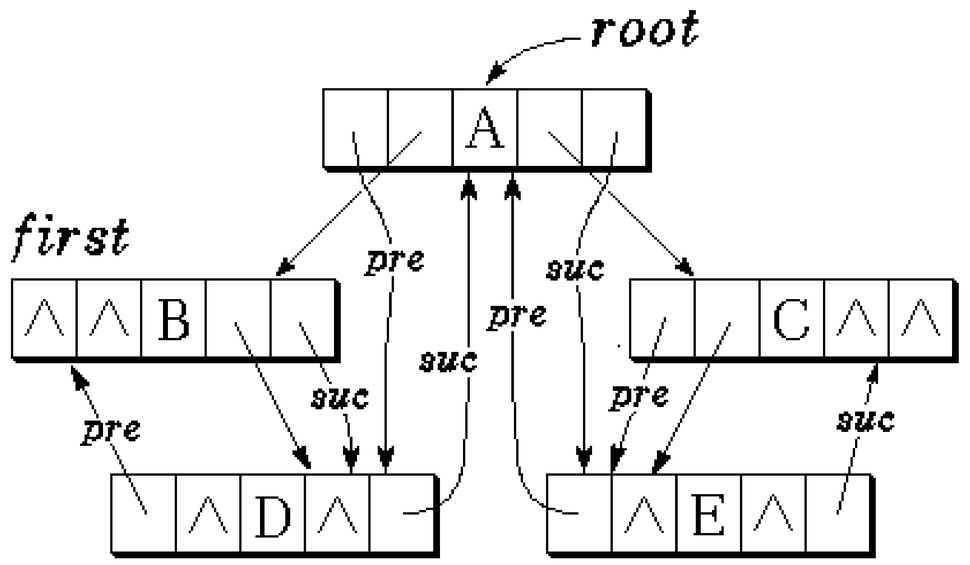
- 二叉树的计数
- 线索化二叉树
- 堆
- 霍夫曼树

线索化二叉树的动机

- 二叉树的前序、中序、后序遍历算法不论是递归或非递归，都隐式或显式地使用了栈
- 如果要经常做遍历，或者需要快速找到某个结点在前序、中序、后序遍历中的前驱或者后继结点，怎么办？
- 方案1：把遍历序列保存起来，每次只要遍历保存的序列即可，但不适合频繁插入删除的二叉树
- 方案2：在原来的二叉链表中增加前驱指针域(Pred)和后继指针域(Succ)，分别指向结点在某种遍历次序下的前驱和后继结点，但是需要额外的存储空间，而原来的leftChild和rightChild指针域中有2n - (n - 1) = n + 1个空指针
- 方案3：线索化二叉树，利用原有的n + 1个空指针域存放指向结点的前驱和后继结点的指针，而这类指针叫做线索(thread)

线索化二叉树的动机

Pred	*leftChild*	*data*	*rightChild*	*Succ*



以中序遍历为例，增加Pred指针和Succ指针的二叉树

中序线索化二叉树

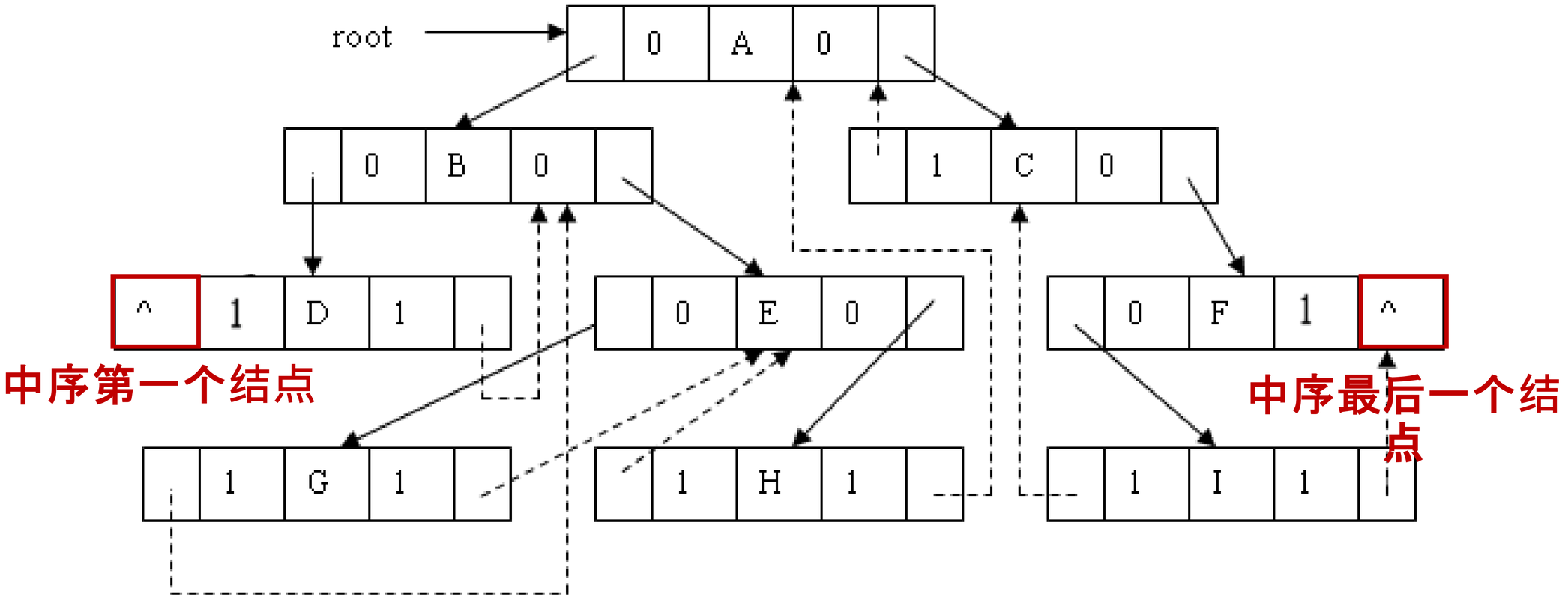
- **如果二叉树的结点k没有左(或右)子结点，k’是k的中序的前驱(或后继)，则置结点k的左(或右)指针为k’的地址**
- **为了与结点k的真正子结点相区别，在结点上需要增加两个字段，即ltag和rtag**

leftChild	ltag	data	rtag	rightChild

- **当结点k的左(或右)指针指向前驱(或后继)时，则置k的ltag(或rtag)为1，而当它们指向真正的子结点时，则置k的ltag(或rtag)为0**
- **只有当结点k是中序的最前(后)结点时，k的左(右)指针才为空，同时k的ltag(rtag)取值为1**

既没有前驱，又没有左子结点

中序线索化二叉树示例



中序序列：DBGEHACIF

中序线索化二叉树的类定义

```
template <class Type> class ThreadTree;
template <class Type> class ThreadNode {
    friend class ThreadTree <Type>;
    private:
        int ltag, rtag;
        ThreadNode <Type> *leftChild, *rightChild;
        Type data;
    public:
        ThreadNode(const Type item) : data(item), leftChild(NULL),
            rightChild(NULL), ltag(0), rtag(0) { }
};
```

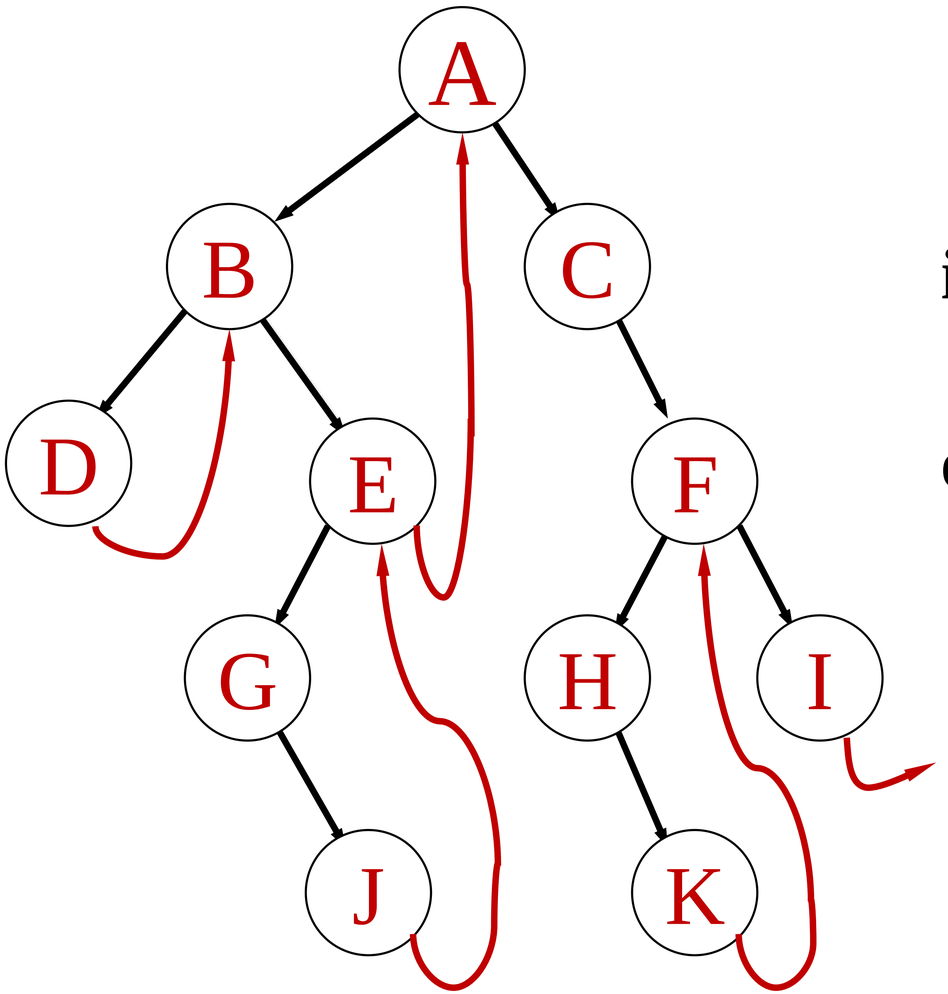
中序线索化二叉树的类定义

```
template <class Type> class ThreadTree {
  protected:
    ThreadNode <Type> * root;  // 树的根指针
    void CreateInThread(ThreadNode <Type> *current,
      ThreadNode <Type> *&pre);  // 中序遍历建立线索二叉树
    ThreadNode <Type> * Parent(ThreadNode <Type> *t);  // 寻找t的父结点
  public:
    ThreadTree( ) : root(NULL) { };  // 构造函数
    void CreateInThread( );
    ThreadNode <Type> * First(ThreadNode <Type> *current);
    ThreadNode <Type> * Last(ThreadNode <Type> *current);
    ThreadNode <Type> * Next(ThreadNode <Type> *current);
    ThreadNode <Type> * Prior(ThreadNode <Type> *current);
    void InOrder(void (*visit) (ThreadNode <Type> *p));
    void PreOrder(void (*visit) (ThreadNode <Type> *p));
    void PostOrder(void (*visit) (ThreadNode <Type> *p));
};
```

寻找以当前结点为根的线索化二叉树在中序下的第一个结点

```
// 返回以current为根的线索化二叉树在中序下的第一个结点
template <class Type> ThreadNode <Type> * ThreadTree <Type> ::
First(ThreadNode <Type> *current) {
    if (current == NULL) return NULL;
    ThreadNode <Type> *p = current;
    while (p->ltag == 0)
        p = p->leftChild;  // 最左下的结点
    return p;
}
```

寻找当前结点在中序下的后继结点



if (current->rtag == 1)

后继为current->rightChild

else // current->rtag == 0

后继为当前结点右子树

的中序下的第一个结点

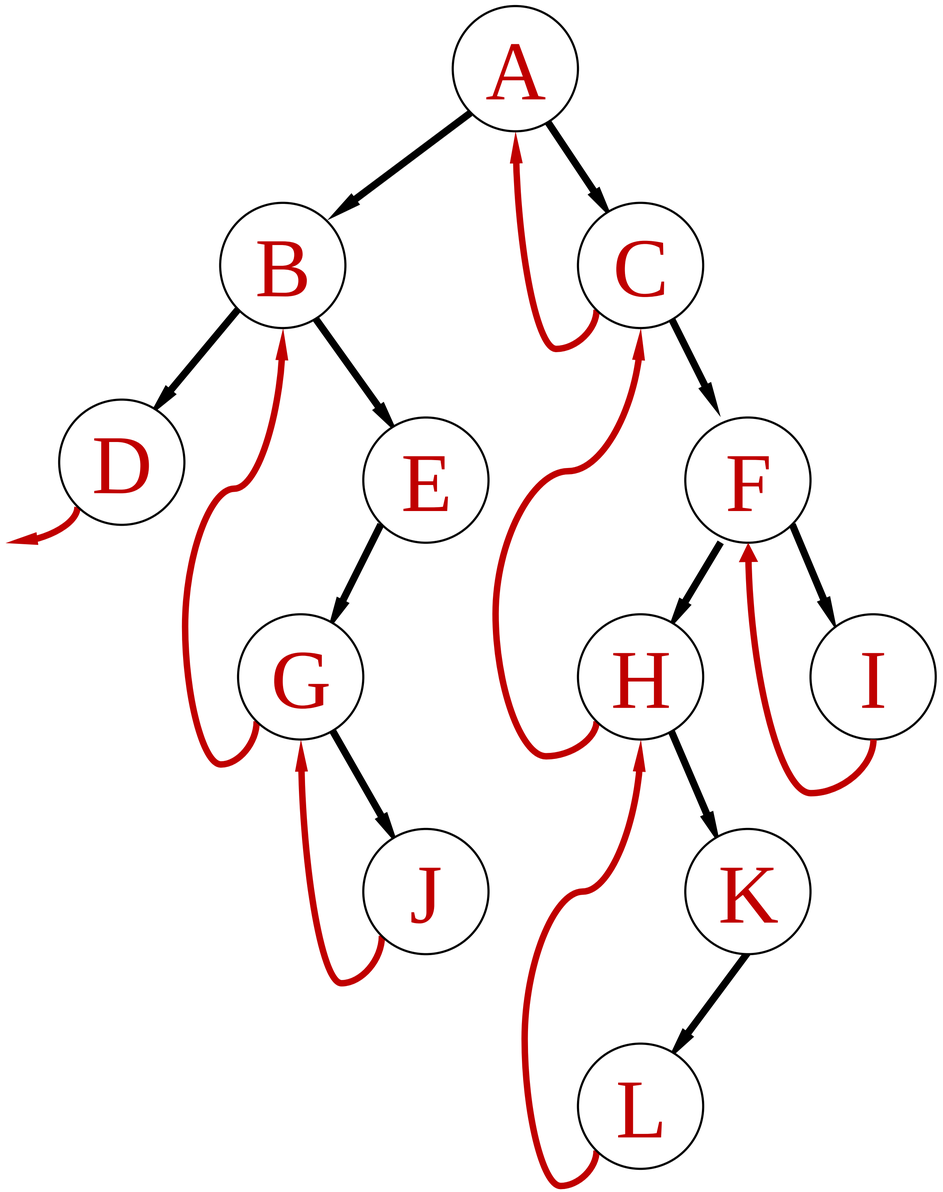
寻找当前结点在中序下的后继结点

```
// 返回在线索化二叉树中结点current在中序下的后继结点
template <class Type> ThreadNode <Type> * ThreadTree <Type> ::
Next(ThreadNode <Type> *current) {
    ThreadNode <Type> *p = current->rightChild;
    if (current->rtag == 0)  // rtag==0，表示有右子女
        return First(p);
    else  // rtag==1，直接返回后继线索
        return p;
}
```

寻找以当前结点为根的线索化二叉树在中序下的最后一个结点

```
// 返回以current为根的线索化二叉树在中序下的最后一个结点
template <class Type> ThreadNode <Type> * ThreadTree <Type> ::
Last(ThreadNode <Type> *current) {
    if (current == NULL) return NULL;
    ThreadNode <Type> *p = current;
    while (p->rtag == 0)
        p = p->rightChild;  // 最右下的结点
    return p;
}
```

寻找当前结点在中序下的前驱结点



if (current->ltag == 1)
**　　前驱为current->leftChild**
else // current->ltag == 0
**　　前驱为当前结点左子树**
**　　中序下的最后一个结点**

寻找当前结点在中序下的前驱结点

```
// 返回在线索化二叉树中结点current在中序下的前驱结点
template <class Type> ThreadNode <Type> * ThreadTree <Type> ::
Prior(ThreadNode <Type> *current) {
    ThreadNode <Type> *p = current->leftChild;
    if (current->ltag == 0)  // ltag==0，表示有左子女
        return Last(p);
    else
        return p;  // ltag==1，直接返回前驱线索
}
```

中序线索化二叉树的中序遍历

- **首先，利用First( )找到二叉树在中序序列下的第一个结点，将其作为当前结点**
- **然后，利用求后继结点的Next( )按中序次序逐个访问，直到二叉树的最后一个结点**
- **在遍历过程中，可以不用栈**

```
template <class Type> void ThreadTree <Type> ::
InOrder(void (*visit) (ThreadNode <Type> *p)) {
    ThreadNode <Type> *p;
    for (p = First(root); p != NULL; p = Next(p))
        visit(p);
}
```

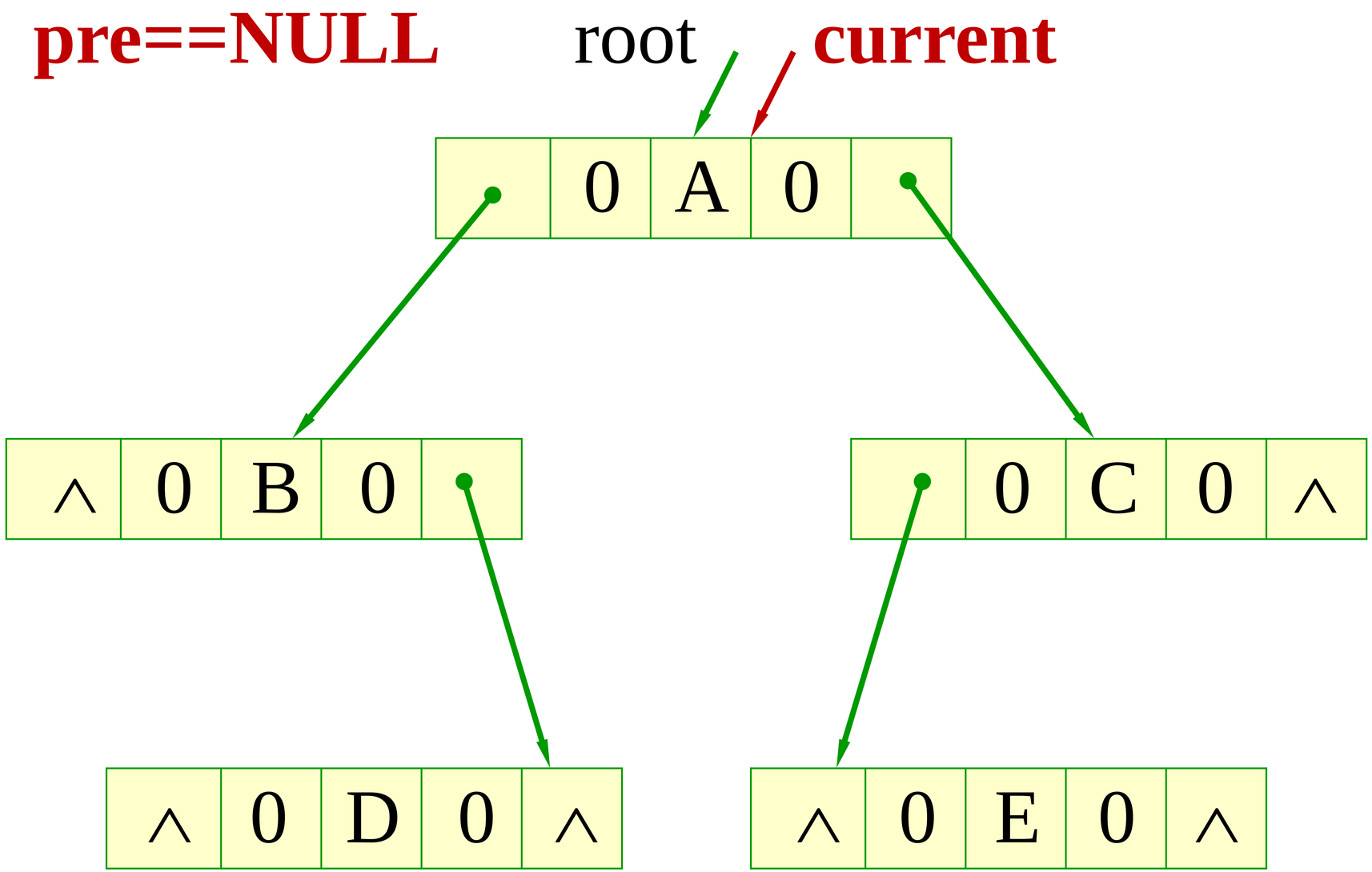
通过中序遍历建立中序线索化二叉树

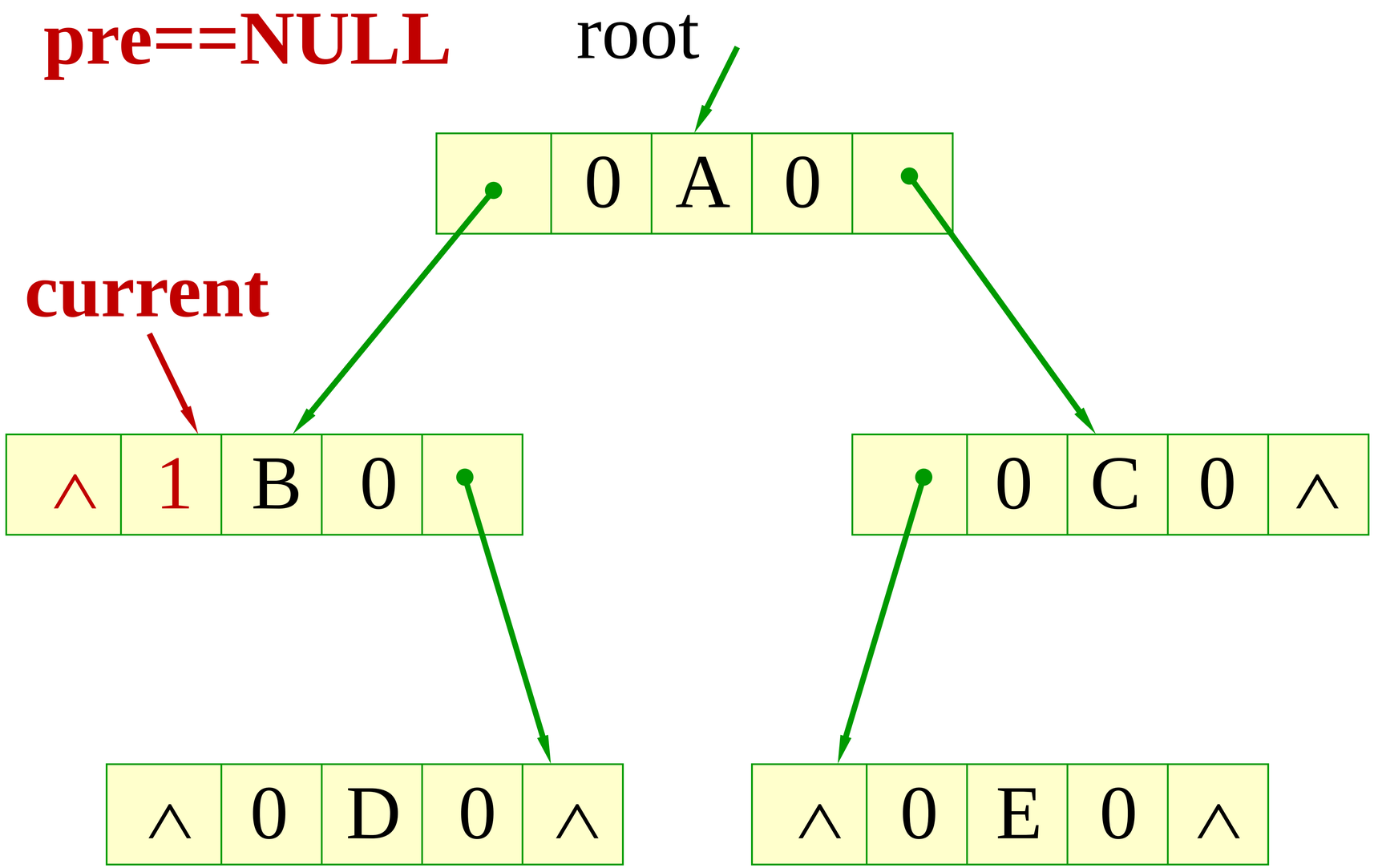
```
template <class Type> void ThreadTree <Type> ::
CreateInThread(ThreadNode <Type> *current, ThreadNode <Type> *&pre) {
    if (current == NULL)  return;
    CreateInThread(current->leftChild, pre);  // 左子树线索化
    if (current->leftChild == NULL) {
        current->leftChild = pre;
        current->ltag = 1;
    }  // 建立当前结点的前驱线索
    if (pre != NULL && pre->rightChild == NULL) {
        pre->rightChild = current;
        pre->rtag=1;
    }  // 建立前驱结点的后继线索
    pre = current;  // 前驱跟上，当前指针向前遍历
    CreateInThread(current->rightChild, pre);  // 右子树线索化
    }
}
```

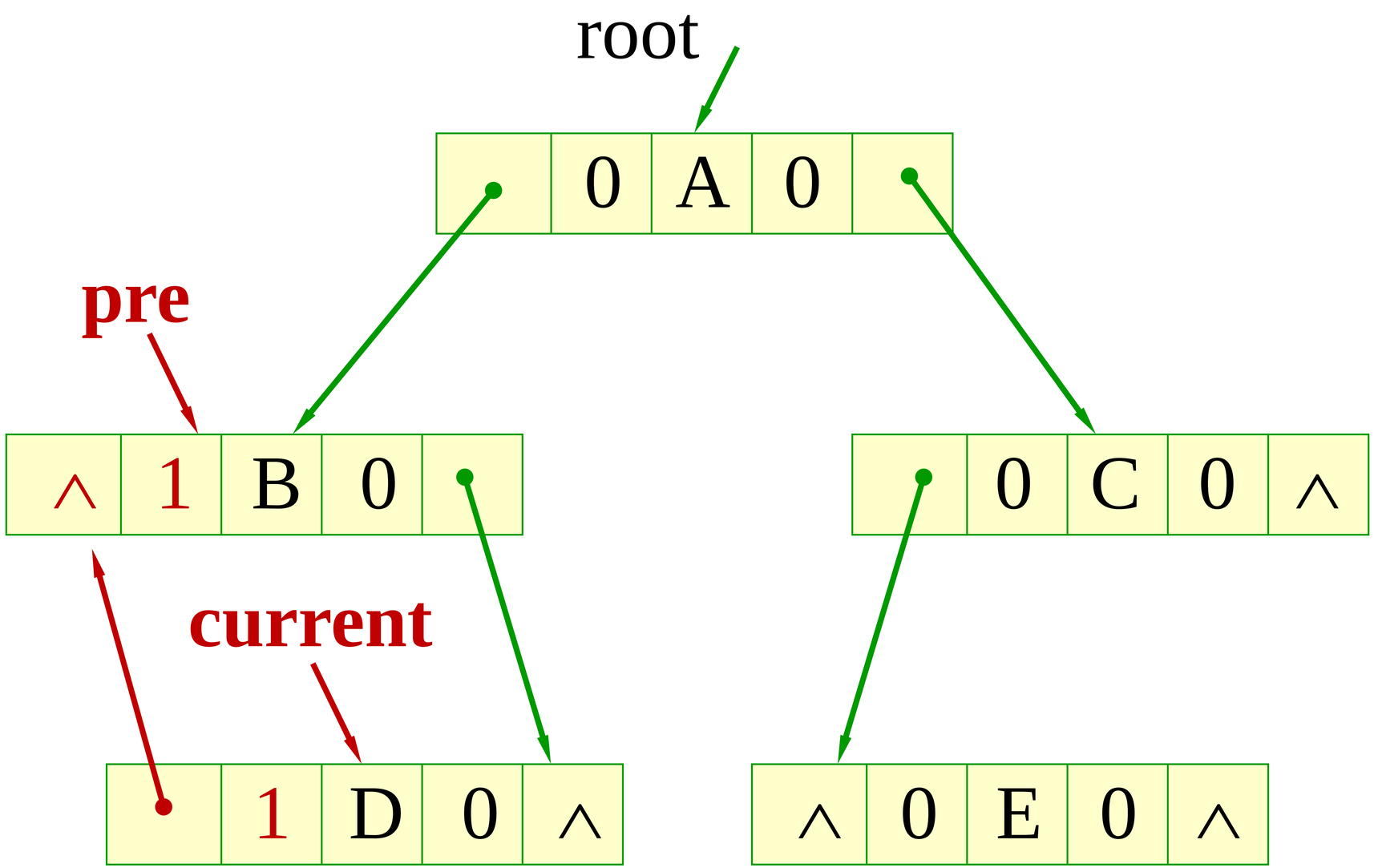
根结点的前驱也是左子树的前驱

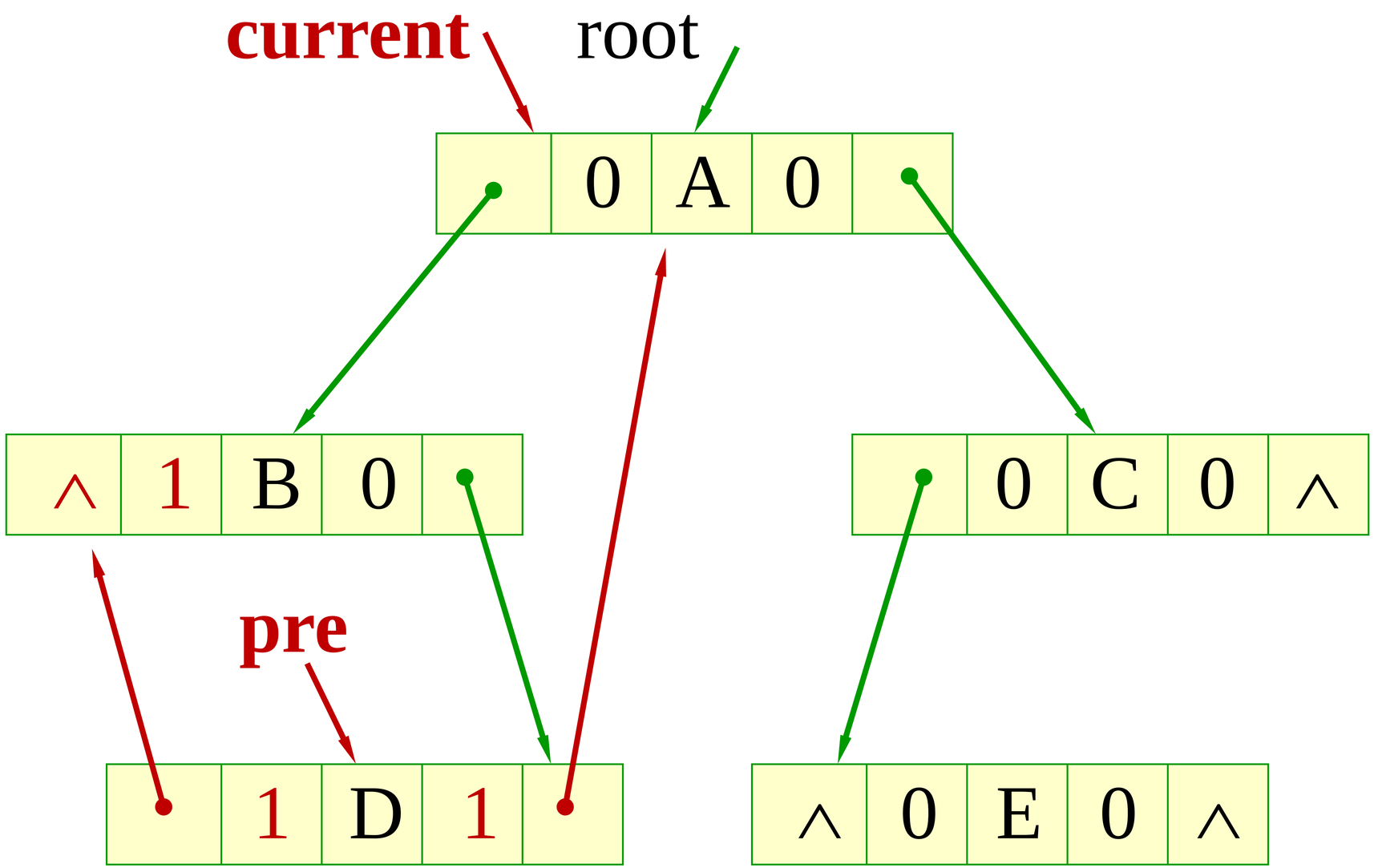
通过中序遍历建立中序线索化二叉树

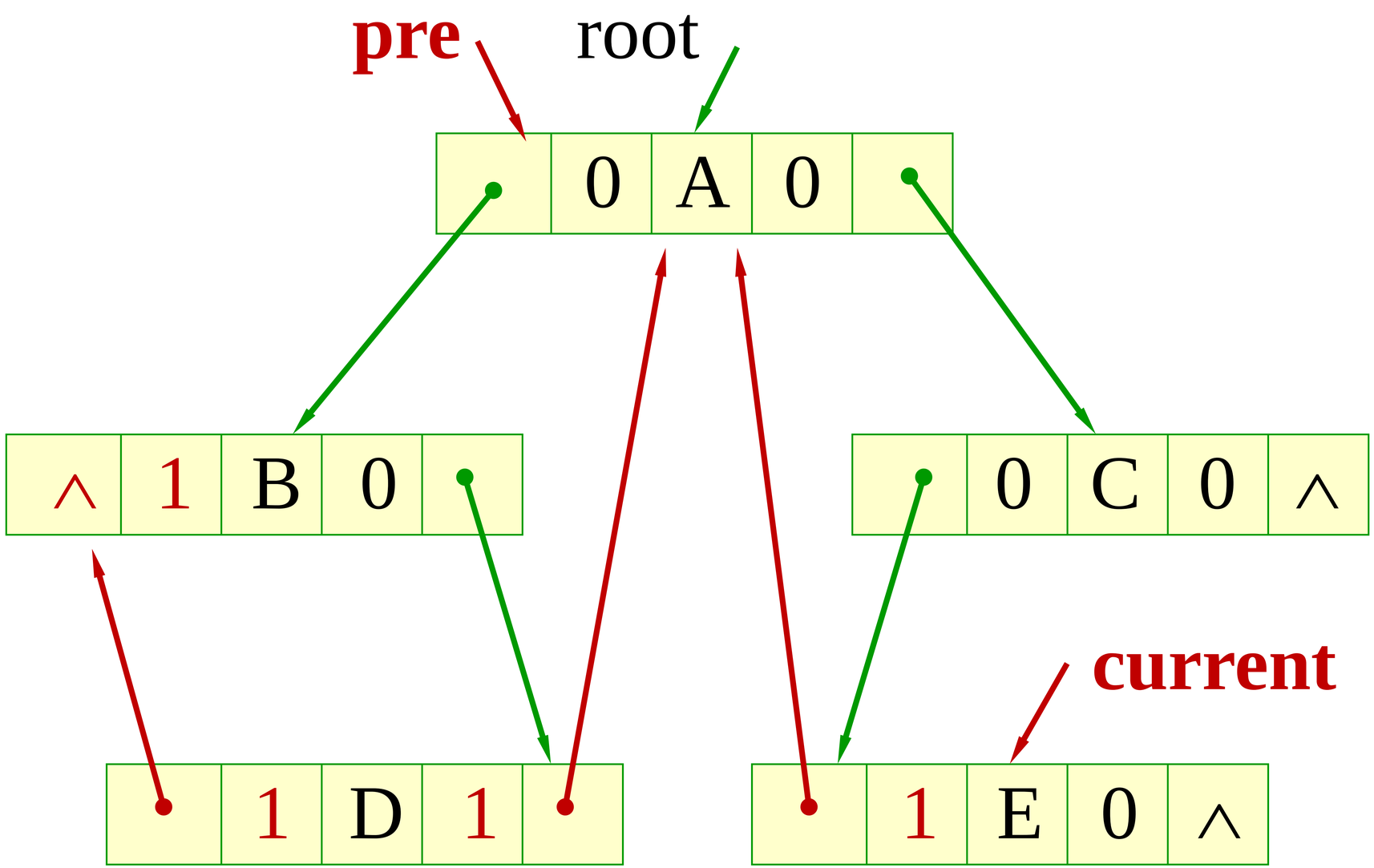
```
template <class Type> void ThreadTree<Type> ::
CreateInThread ( ) {
    ThreadNode<Type> *pre = NULL;  // 前驱指针
    if (root != NULL) {  // 非空二叉树，线索化
        CreateInThread (root, pre);  // 中序遍历线索化二叉树
        pre->rightChild = NULL;
        pre->rtag = 1;  // 后处理，中序最后一个结点
    }
}
```

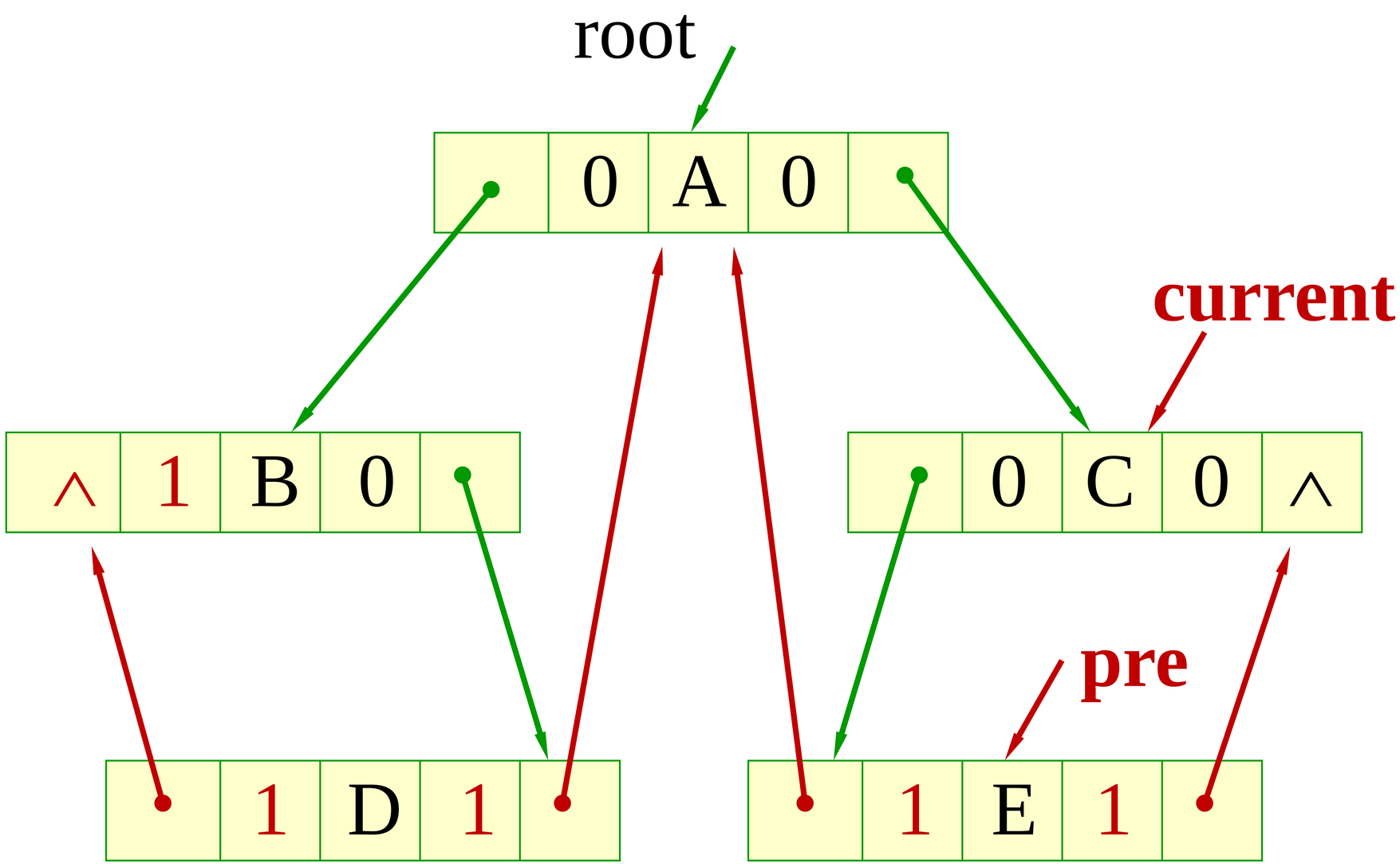

pre==NULL
root
current
0 A 0
∧ 0 B 0
0 C 0 ∧
∧ 0 D 0 ∧
∧ 0 E 0 ∧

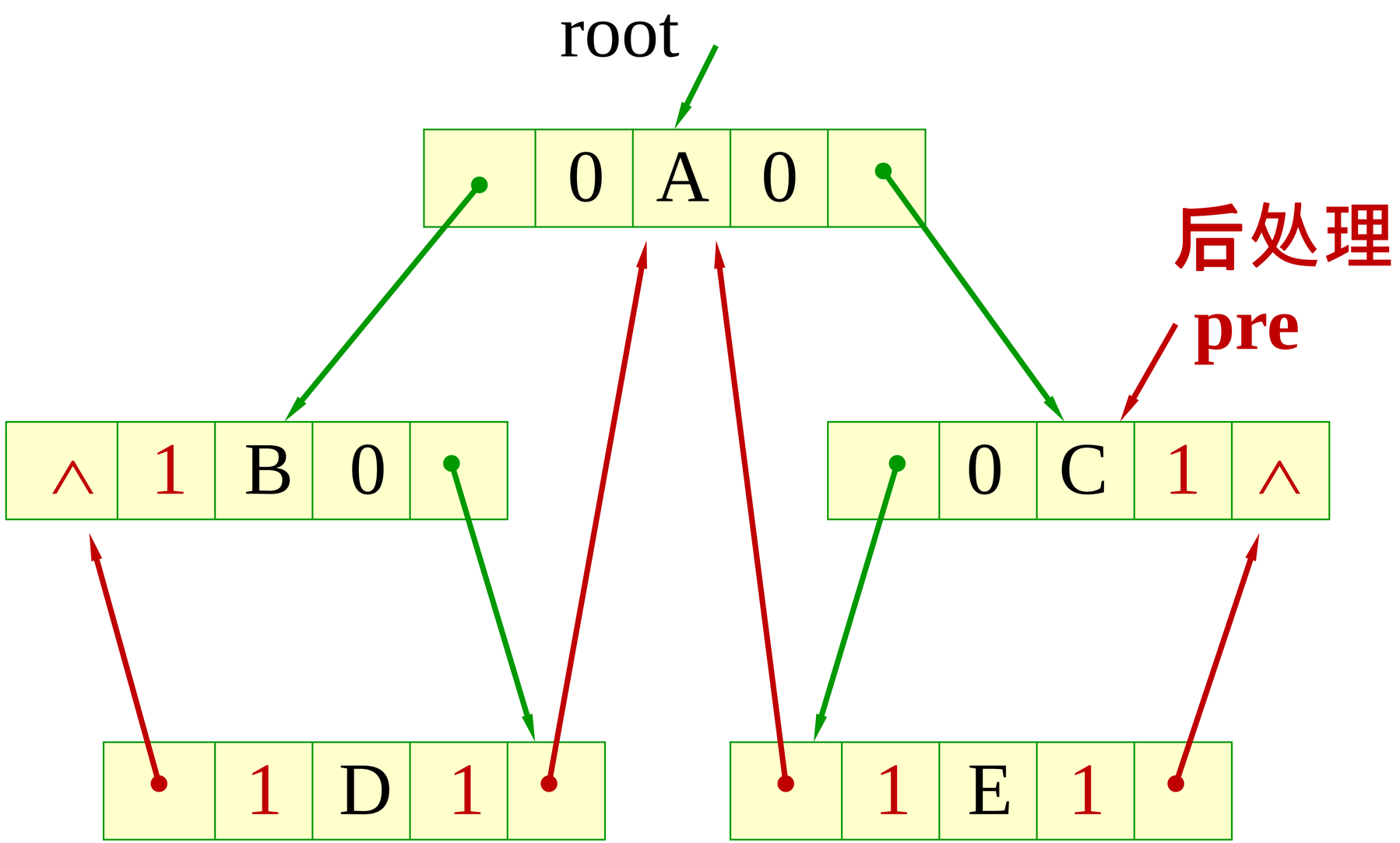

pre==NULL
root
0 A 0
current
∧ 1 B 0
0 C 0 ∧
∧ 0 D 0 ∧
∧ 0 E 0 ∧


root
0
A
0
pre
^
1
B
0
0
C
0
^
current
1
D
0
^
^
0
E
0
^


current
root
0
A
0
^
1
B
0
0
C
0
^
pre
1
D
1
^
0
E
0
^

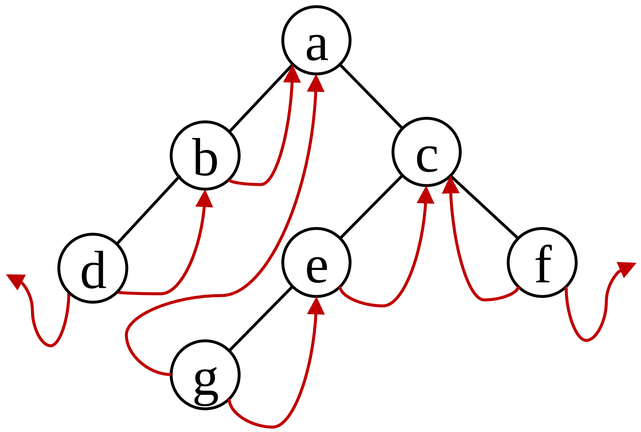

pre
root
0
A
0
^
1
B
0
0
C
0
^
1
D
1
current
1
E
0
^


root
0 A 0
current
^ 1 B 0
0 C 0 ^
1 D 1
pre
1 E 1


root
0 A 0
后处理
pre
^ 1 B 0
0 C 1 ^
1 D 1
1 E 1

练习1

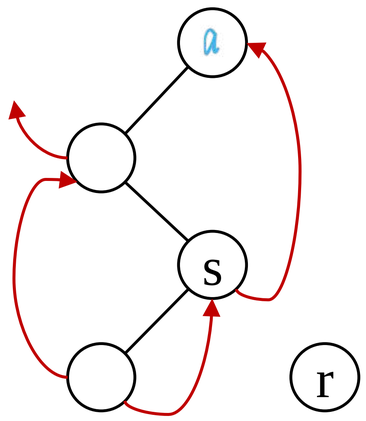
- 将以下的二叉树按照中序线索化



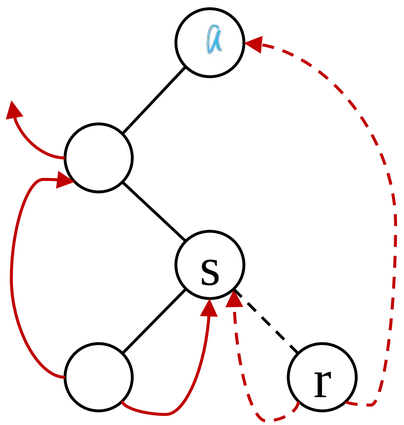
中序序列：dbagecf

中序线索化二叉树的插入操作

- **当一个新结点r作为结点s的右子女插入时**
  - **若s的rightChild是线索，则s的后继线索成为r的后继线索，r的前驱线索指向s，r成为s的右子女**
  - **若s的rightChild是右子女指针，则s的右子树成为r的右子树，且该子树中中序下的第一个结点的前驱线索指向r，r的前驱线索指向s，r成为s的右子女**

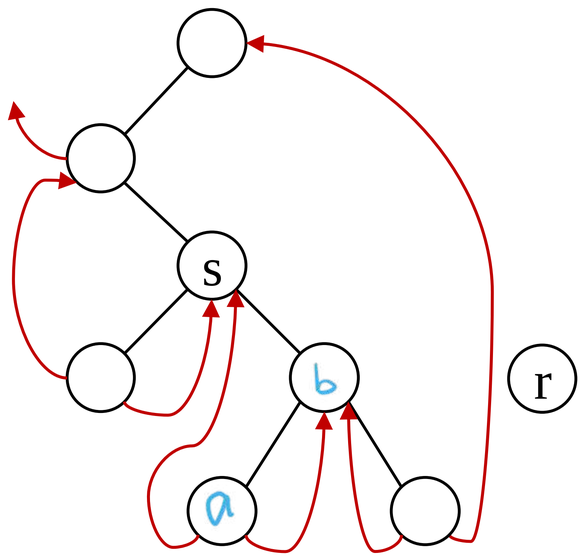


s a



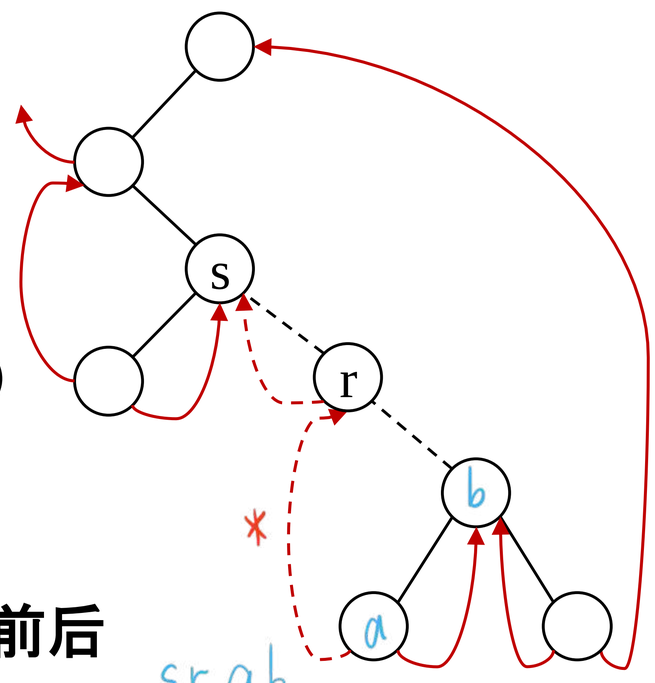
s r a

插入前后



s a b

插入前后

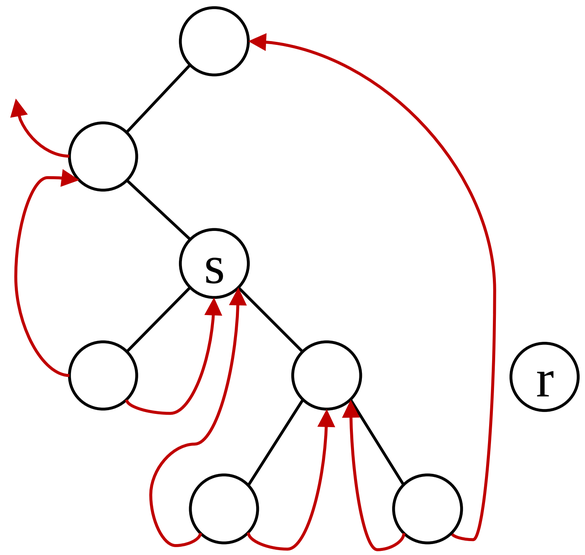


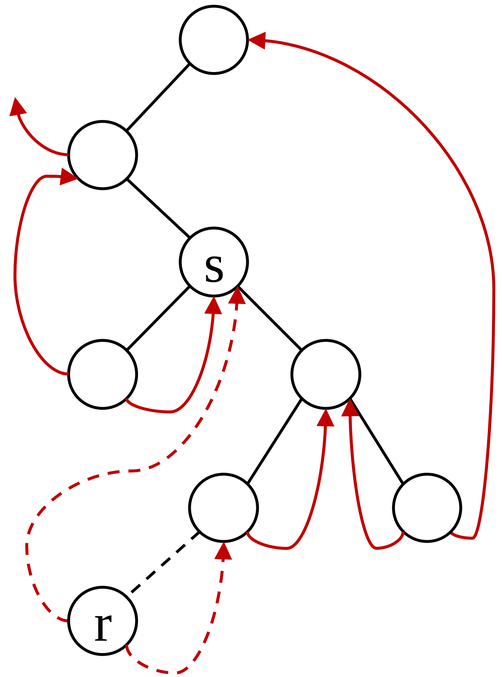
s r a b

中序线索化二叉树的插入操作

```
// 将r当做s的右子女插入
template <class Type> void ThreadTree <Type> ::
InsertRight(ThreadNode <Type> *s, ThreadNode <Type> *r) {
    // s的右子女指针或后继线索传给r
    r->rightChild = s->rightChild; r->rtag = s->rtag;
    // r的leftchild成为指向s的前驱线索
    r->leftChild = s;  r->ltag = 1;
    // r成为s的右子女
    s->rightChild = r;  s->rtag = 0;
    if (r->rtag == 0) {  // s原来有子女
        // 在s原来的右子树中找中序下第一个结点
        ThreadNode <Type> *temp = First(r->rightChild);
        temp->leftChild = r;  // 建立指向r的前驱线索
    }
}
```

中序线索化二叉树的插入操作





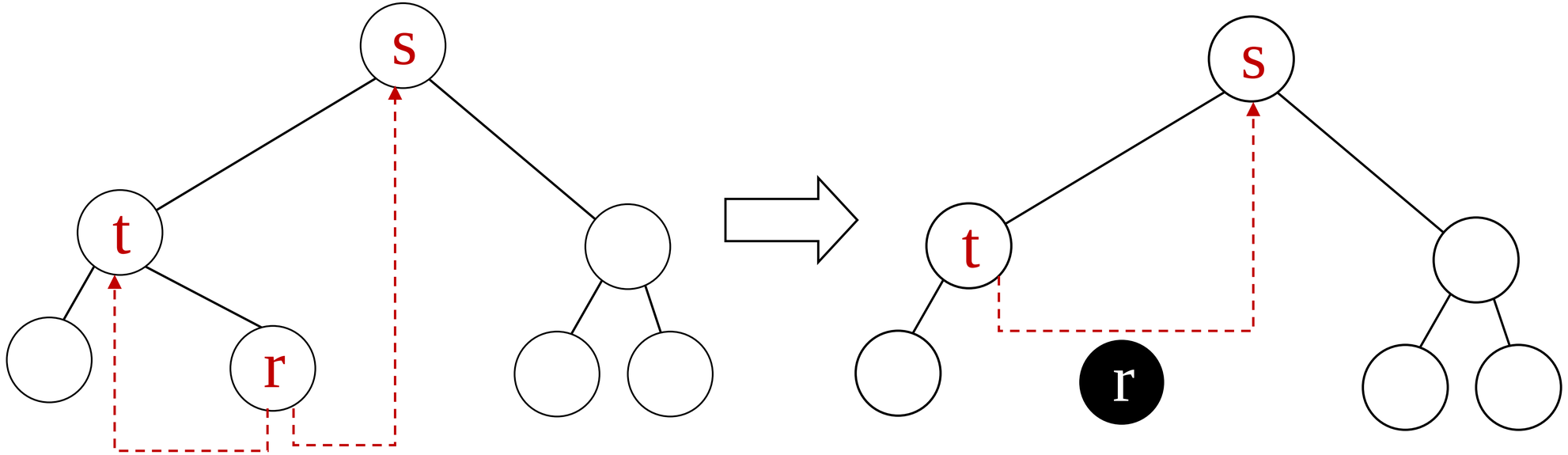
若s的rightChild是右子女指针，另外一种插入方式是把r当作s的后继的左子女插入，同样能保证在中序中r位于s之后

中序线索化二叉树的插入操作

```
// 将r当做s的右子女插入
template <class Type> void ThreadTree <Type> ::
InsertRight(ThreadNode <Type> *s, ThreadNode <Type> *r) {
    if (s->rtag == 1) {
        r->rightChild = s->rightChild; r->rtag = s->rtag; // s的后继线索传给r
        r->leftChild = s;  r->ltag = 1;  // r的leftchild成为指向s的前驱线索
        s->rightChild = r;  s->rtag = 0;  // r成为s的右子女
    } else {  // r作为s的后继的左子女插入
        ThreadNode <Type> *temp = Next(s);
        // s的后继的前驱传给r的前驱
        r->leftChild = temp->leftChild; r->ltag = temp->ltag;
        r->rightChild = temp; t->rtag = 1;  // r的后驱线索指向s的后继
        temp->leftChild = r; temp->ltag = 0;  // r成为s的后继的左子女
    }
}
```

中序线索化二叉树的删除操作

- **以删除t的右子女r为例，分4种情况，情况1：**
  - **r是叶子结点，没有左右子树：r的后继s如果存在则s的左孩子非空，故不需要调整s的前驱**
  - **t->rightChild = r->rightChild; t->rtag = r->rtag;**

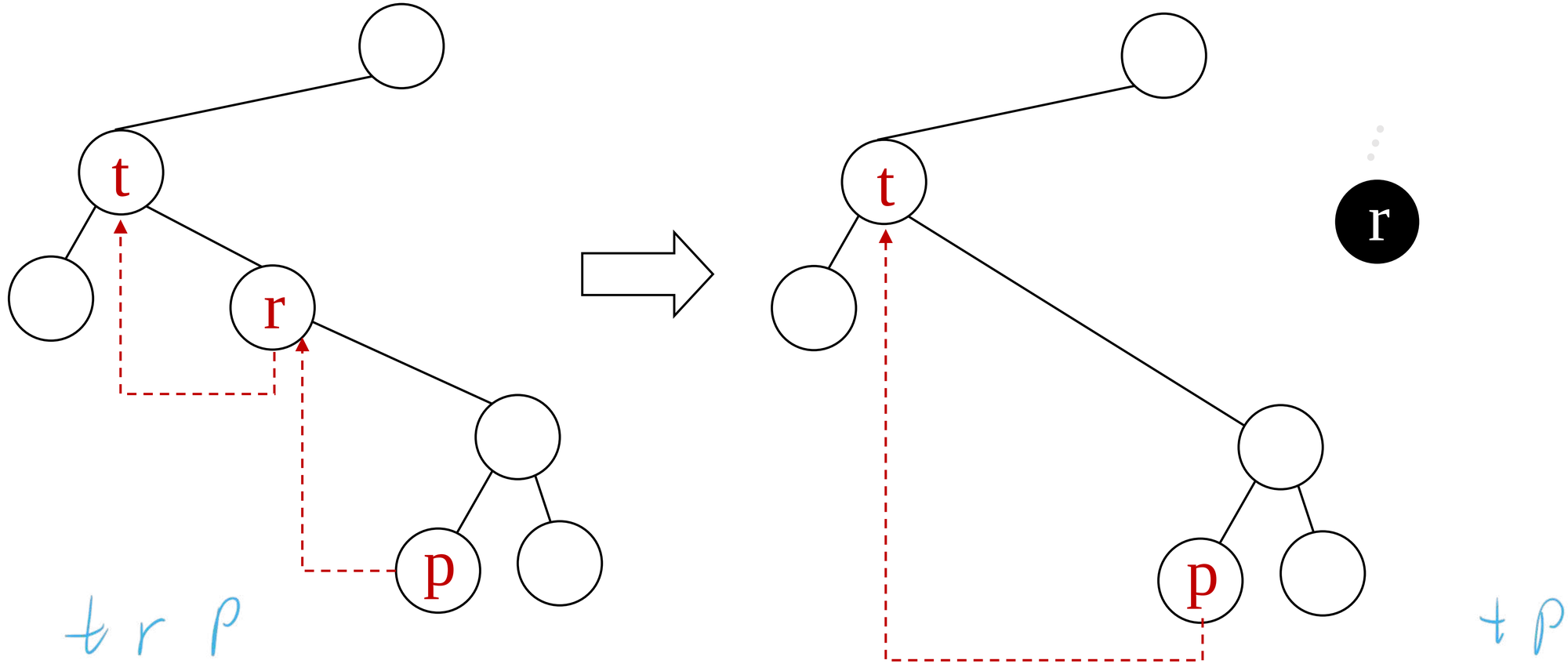


t r s

t s

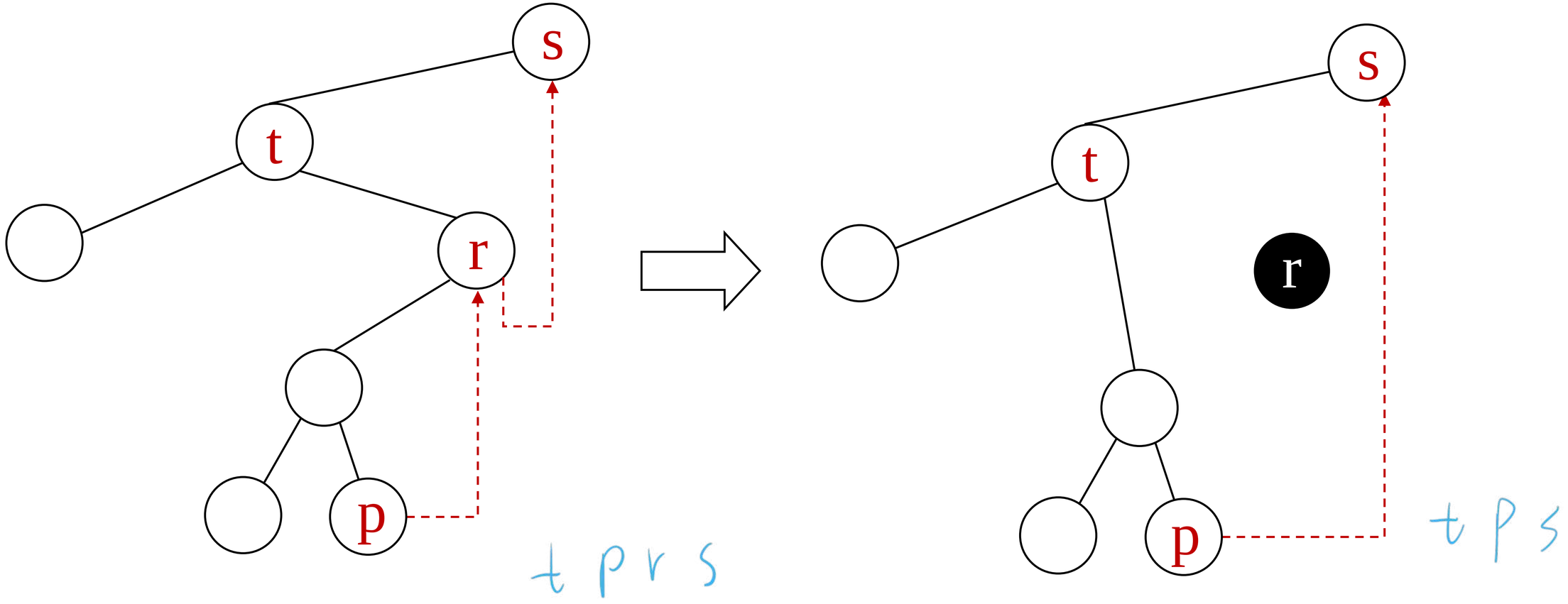
中序线索化二叉树的删除操作

- **以删除t的右子女r为例，分4种情况，情况2**
  - **r的左孩子为空：要让r的右子树的中序第一个结点p记住r的前驱**
  - **ThreadNode<Type> *p = First(r->rightChild);**
  - **p->leftChild = r->leftChild; t->rightChild = r->rightChild;**



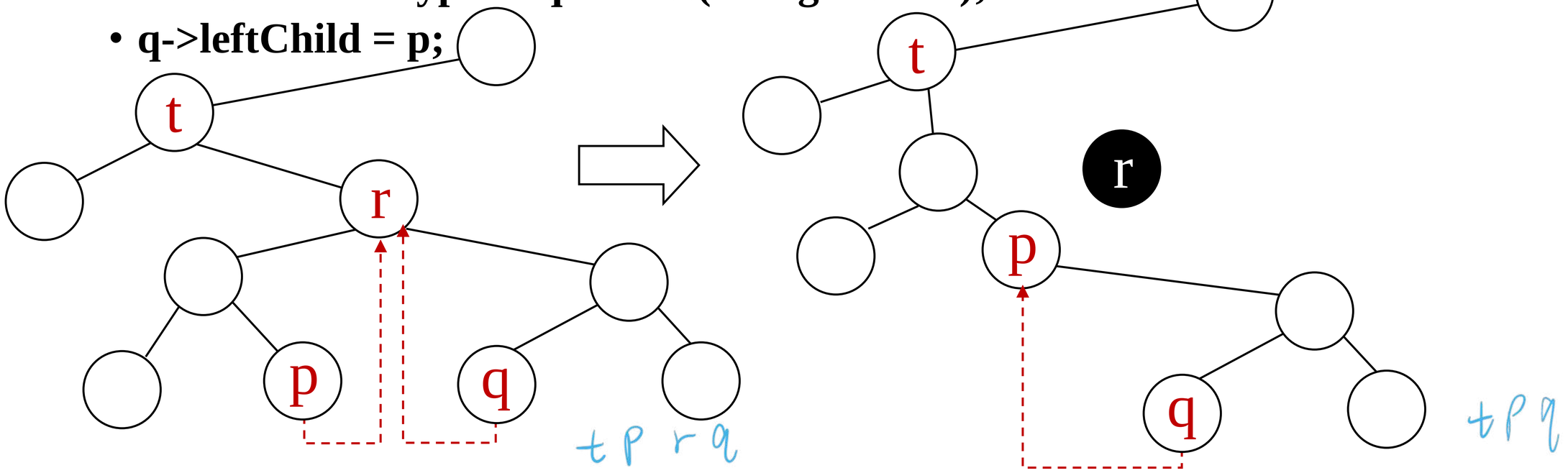
中序线索化二叉树的删除操作

- **以删除t的右子女r为例，分4种情况，情况3**
  - **r的右孩子为空：要让r的左子树的中序最后一个结点p记住r的后继**
  - **ThreadNode<Type> *p = Last(r->leftChild);**
  - **p->rightChild = r->rightChild; t->rightChild = r->leftChild;**



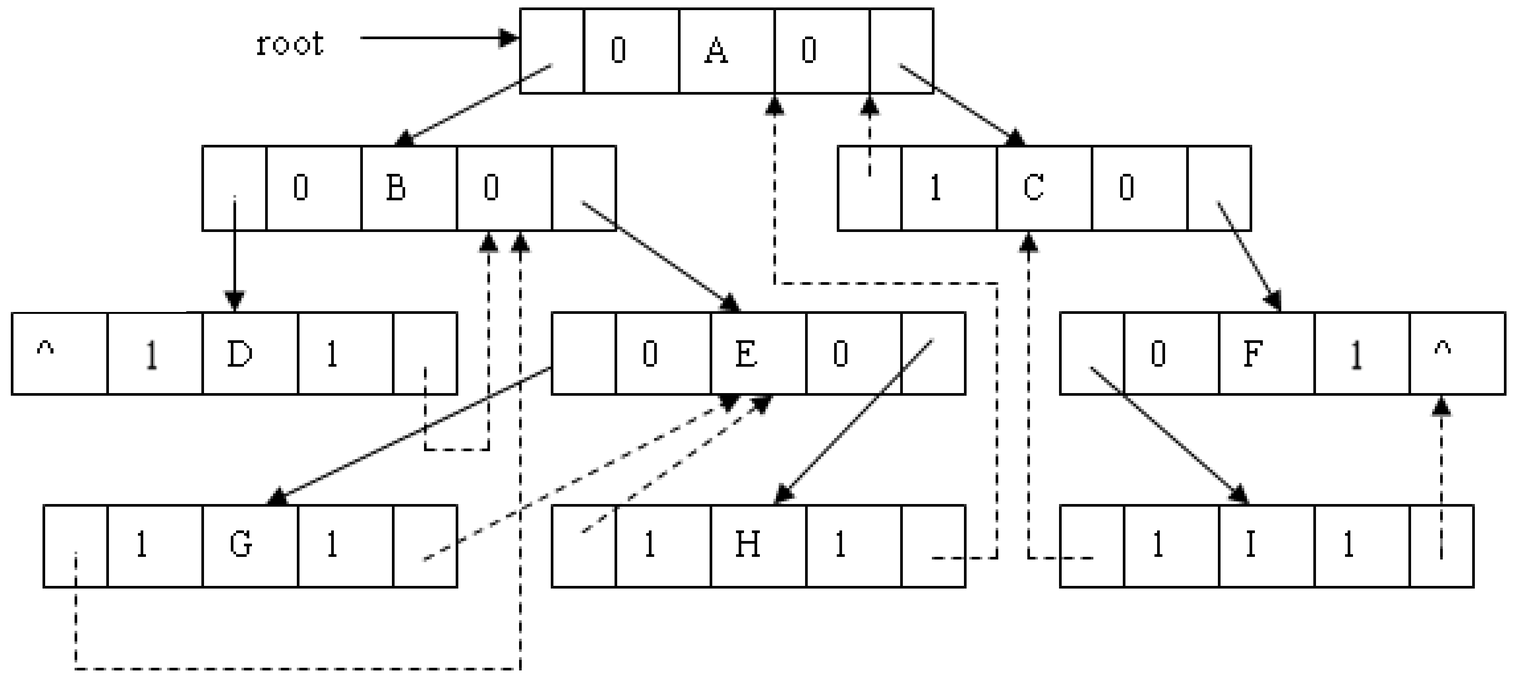
中序线索化二叉树的删除操作

- **以删除t的右子女r为例，分4种情况，情况4**
  - **r有左右子树：把r的左子树上升到t的右子女位置，把r的右子树接到r的左子树的中序最后一个结点p的右子树**
  - **ThreadNode<Type> *p = Last(r->leftChild);**
  - **p->rightChild = r->rightChild; p->rtag = r->rtag;**
  - **t->rightChild = r->leftChild;**
  - **ThreadNode<Type> *q = First(r->rightChild);**
  - **q->leftChild = p;**



线索信息的利用：前序遍历

- **前序序列中的第一个结点为二叉树的根，故从根结点开始，访问当前结点**
  - **若当前结点有左子女，则前序下的后继结点为左子女结点**
  - **否则，若当前结点有右子女，则前序下的后继结点为右子女结点**
  - **否则，当前结点是叶结点，则沿着中序后继线索走到一个有右子女结点的结点，该右子女结点为当前结点的前序下的后继结点**

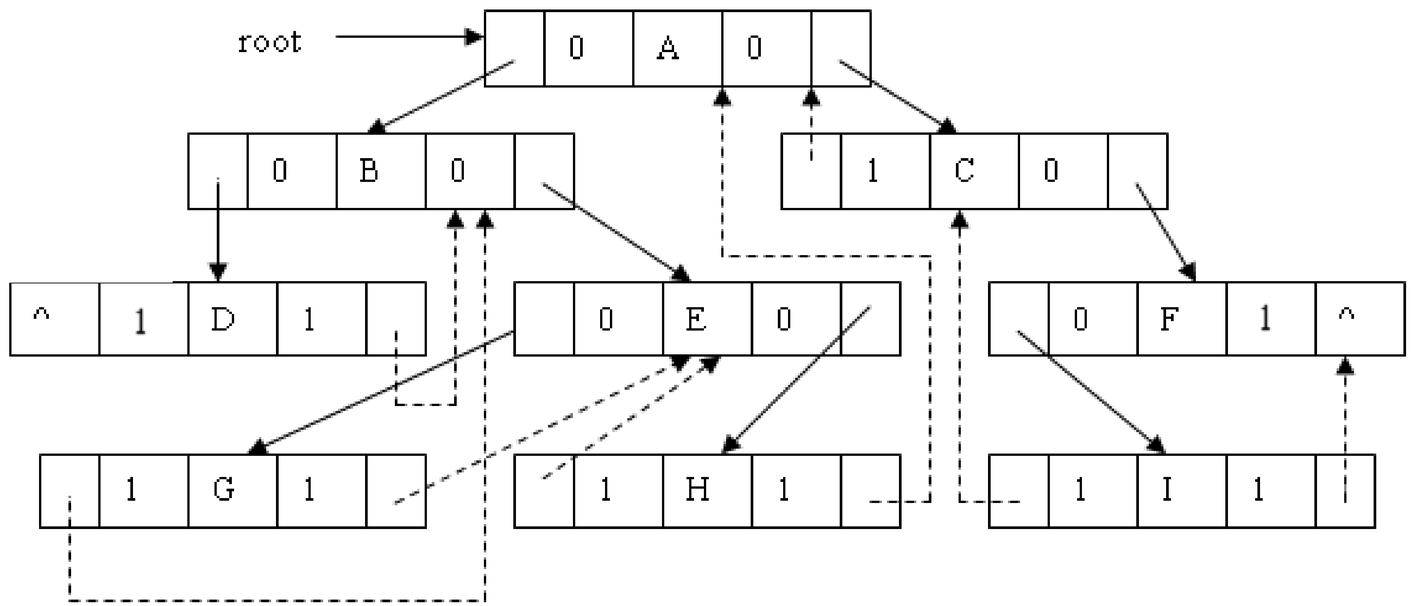


线索信息的利用：前序遍历

```
template <class Type> void ThreadTree <Type> ::
PreOrder(void (*visit) (ThreadNode <Type> *p)) {
    ThreadNode <Type> *p = root;
    while (p != NULL) {
        visit(p);
        if (p->ltag == 0)  p = p->leftChild;  // 有左子女，即为后继
        else if (p->rtag == 0)  p = p->rightChild;  // 有右子女，即为后继
        else {
            while (p != NULL && p->rtag == 1)  // 沿后继线索检测
                p = p->rightChild;  // 直到有右子女的结点
            if (p != NULL)  p = p->rightChild;  // 右子女即为后继
        }
    }
}
```

线索信息的利用：后序遍历

- 从根出发，沿着左子女链一直找下去，找到左子女不再是左子女指针的结点，再找到该结点的右子女，在以此结点为根的子树上再重复上述过程，直到叶结点为止
- 从此结点开始后序遍历，先找到当前结点的父结点
- 若当前结点是父结点的右子女，或者当前结点是父结点的左子女但该父结点没有右子女，则后序下的后继结点为该父结点
- 否则，在当前结点的右子树(若存在)上重复执行上面的操作

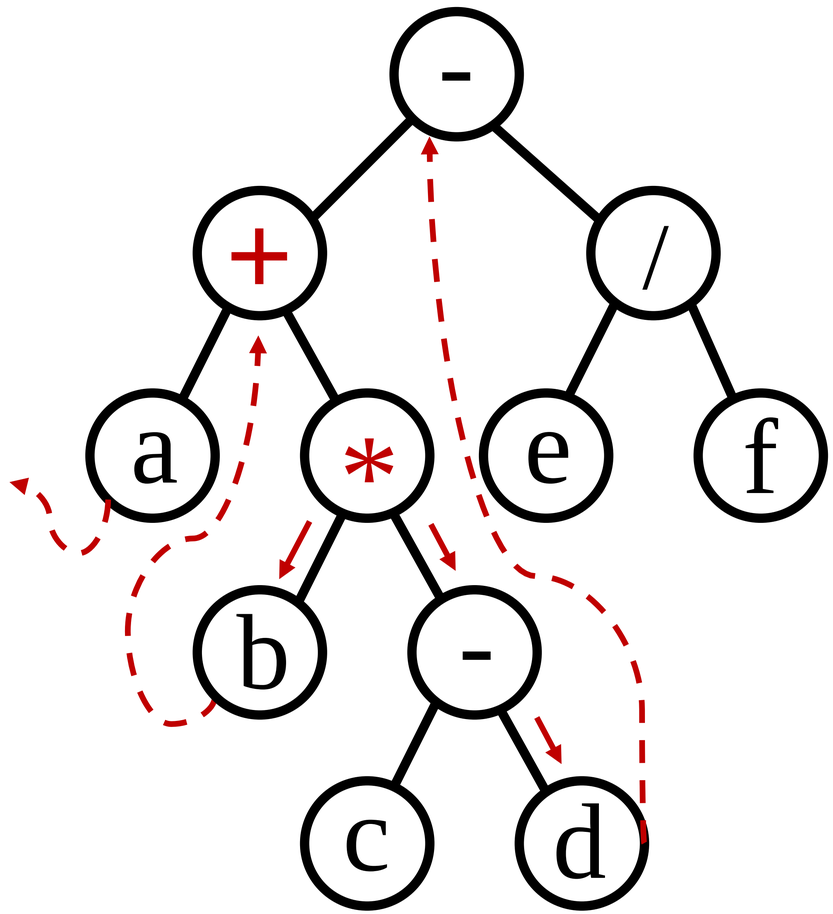


线索信息的利用：后序遍历

```
template <class Type> void ThreadTree <Type> ::
PostOrder(void (*visit) (ThreadNode <Type> *p)) {
    ThreadNode <Type> *t = root;
    while (t->ltag == 0 || t->rtag == 0)  // 寻找后序下的第一个结点
        if (t->ltag == 0)  t = t->leftChild;  else if (t->rtag == 0)  t = t->rightChild;
    visit(t);
    ThreadNode <Type> *p;
    while ((p = Parent(t)) != NULL) {
        if (p->rightChild == t || p->rtag == 1)  t = p;  // p是后序后继
        else {
            t = p->rightChild;  // 开始遍历p的右子树
            while (t->ltag == 0 || t->rtag == 0)
                if (t->ltag==0)  t=t->leftChild;  else if (t->rtag==0)  t=t->rightChild;
        }
        visit(t);
    }
}
```

线索信息的利用：寻找父结点

- 从当前结点(沿左子女链)走到树上层的一个中序前驱(不一定是直接前驱)，然后向右下找父结点
- 从当前结点(沿右子女链)走到树上层的一个中序后继(不一定是直接后继)，然后向左下找父结点
- 只从一个方向找，未必找得到
  - 中序下第一个结点前驱为空，中序下最后一个结点后继为空
- 根结点在一开始就要排除，因为没有双亲结点



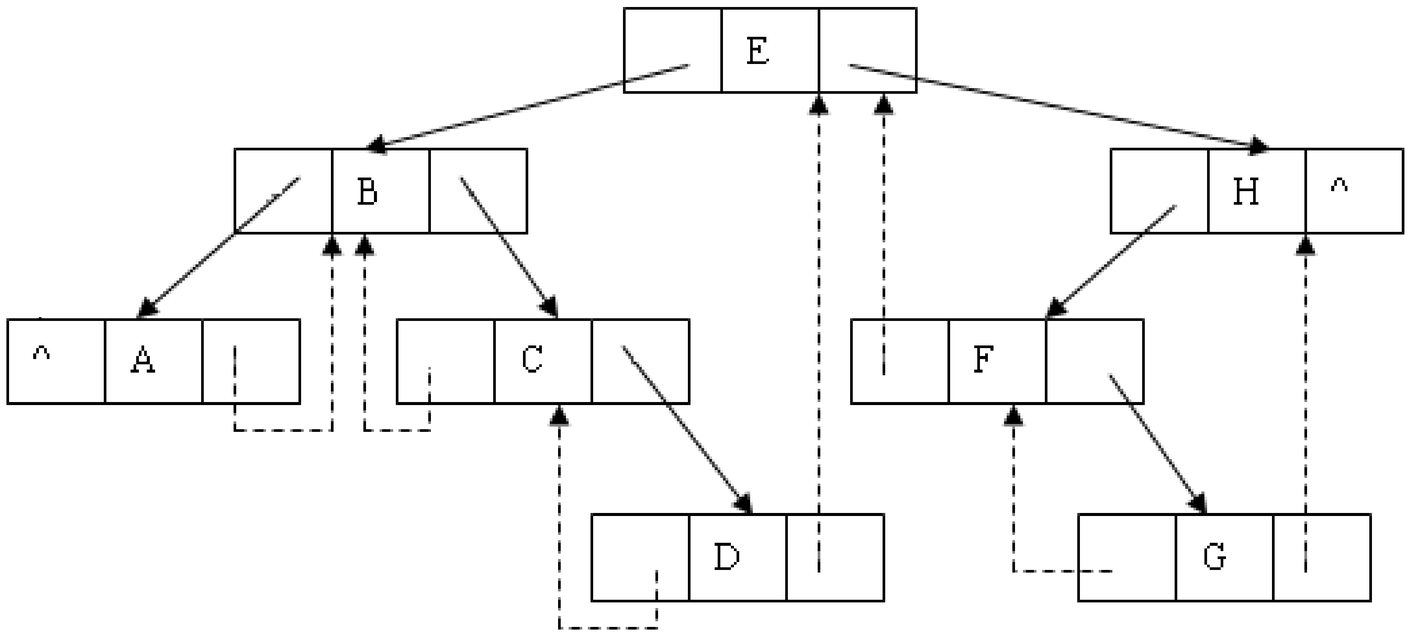
线索信息的利用：寻找父结点

```
template <class Type> ThredNode <Type> * ThreadTree <Type> ::
Parent(ThreadNode <Type> *t) {
    ThreadNode <Type> *p;
    if (t == root)  return NULL;  // 根结点无父结点
    for (p = t; p->ltag == 0; p = p->leftChild);
    if (p->leftChild != NULL)  // 树上层的一个中序前驱
        for (p = p->leftChild; p != NULL && p->leftChild != t &&
            p->rightChild != t; p = p->rightChild);  // 往右下找
    if (p == NULL || p->leftChild == NULL) {  //找到底未找到或直接无法找
        for (p = t; p->rtag == 0; p = p->rightChild);  // 树上层的一个中序后继
        for (p = p->rightChild; p != NULL && p->rightChild != t &&
            p->leftChild != t; p = p->leftChild);  // 往左下找
    }
    return p;
}
```

线索信息的利用：线索排序

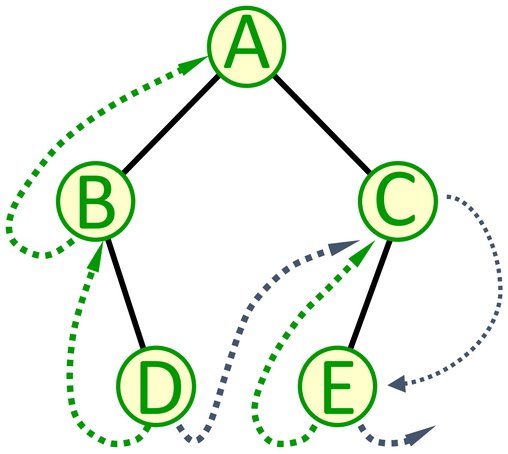
- 首先，用给定的n个结点建造一棵线索树，使树中每个结点的值大于其非空左子树中所有结点的值，而小于其非空右子树中所有结点的值，称这样的树为线索排序树
- 然后，按中序遍历线索排序树，得到n个结点的新序列，它是由小到大排好序的，即线索排序

线索信息的利用：线索排序



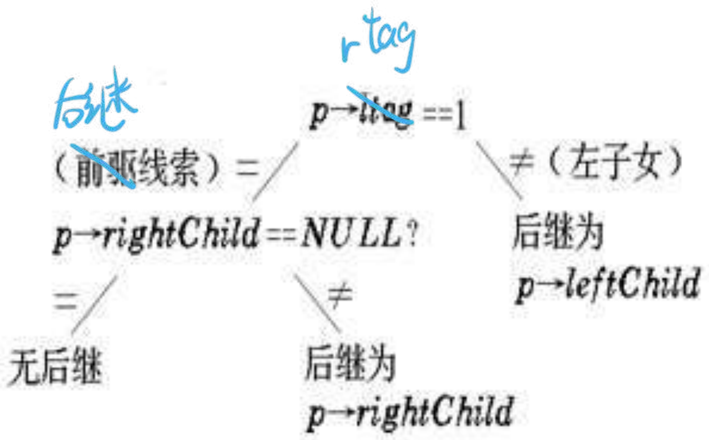
根据给定序列[E, B, A, C, D, H, F, G]， 创建线索排序树

前序线索化二叉树

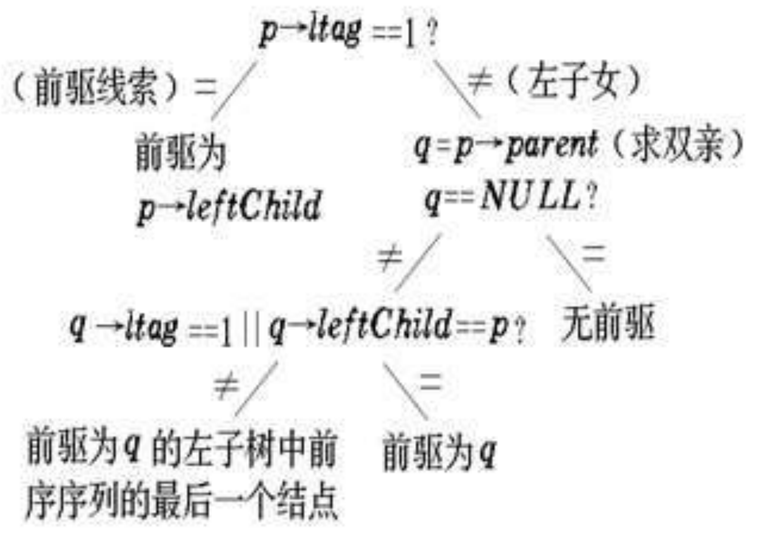


前序序列

A B D C E

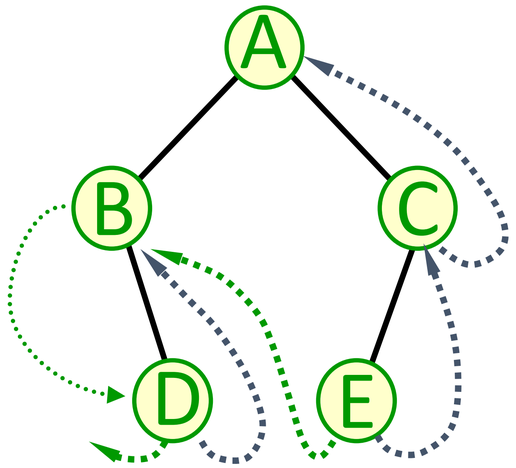


（a）求结点 p 的后继



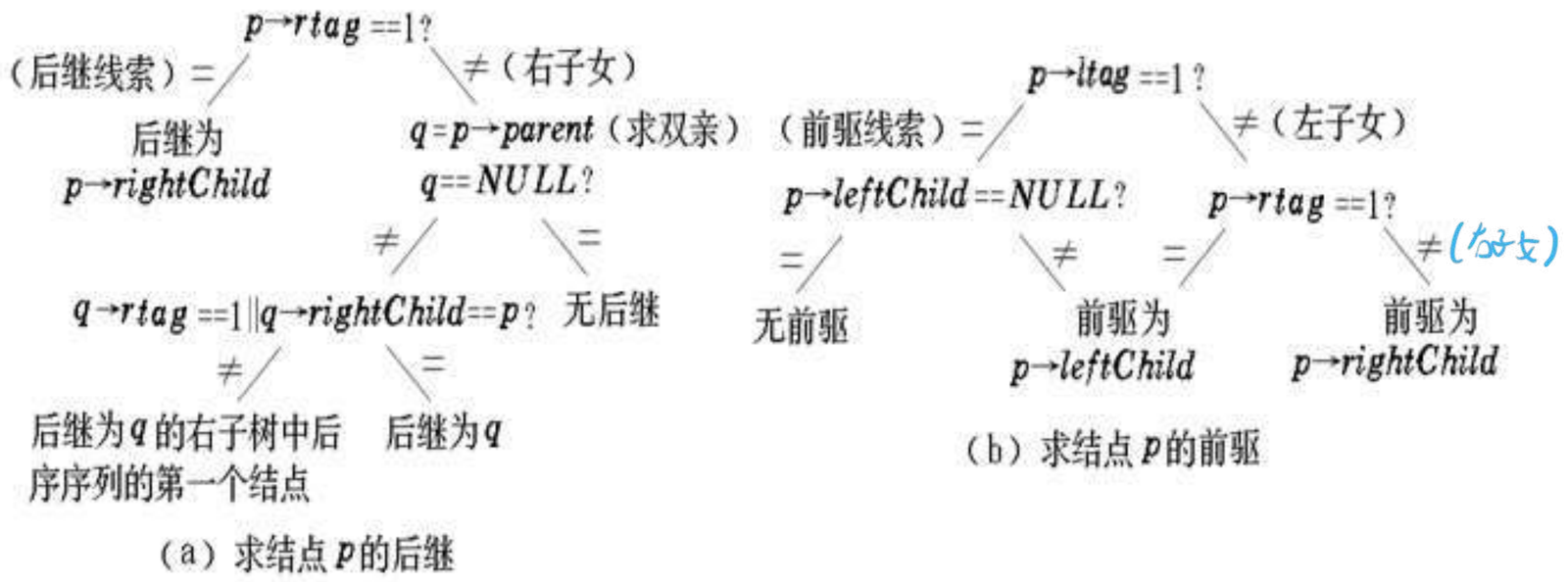
（b）求结点 p 的前驱

后序线索化二叉树



后序序列

D B E C A



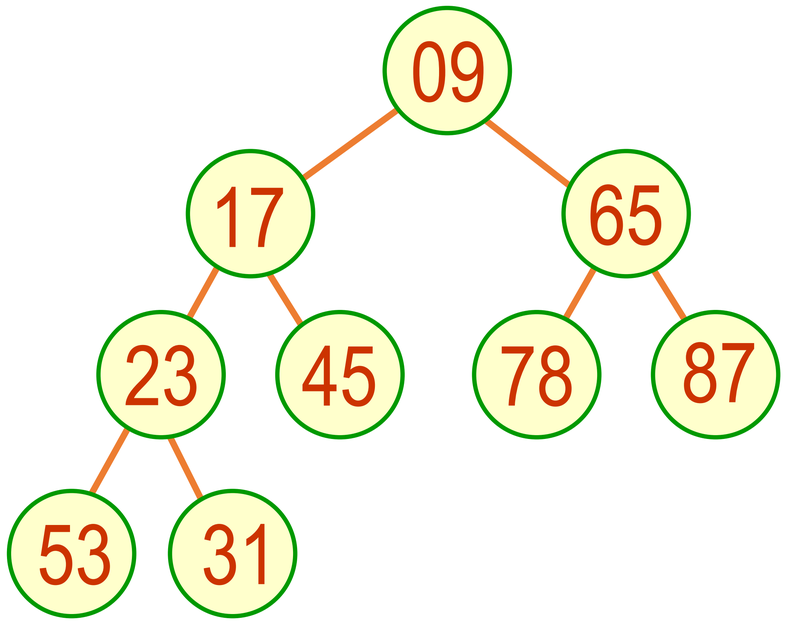
06 树

- 二叉树的计数
- 线索化二叉树
- **堆**
- 霍夫曼树

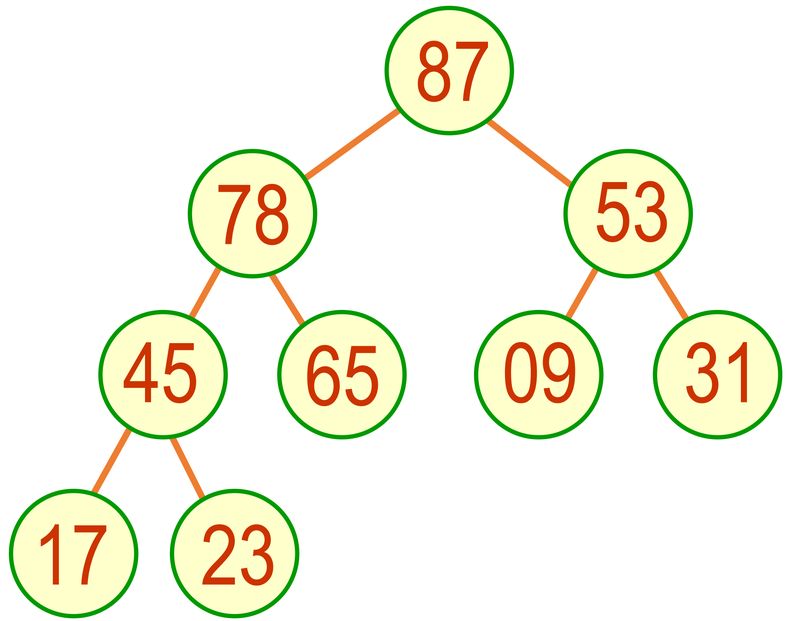
堆的定义

- 假设二叉树T是一棵完全二叉树，如果T中的任一结点的值不小于(不大于)它的任一子结点的值(如果存在子结点的话)，那么称T是一个**最大(最小)堆**(heap)
- 如果具有n个结点的完全二叉树T是一个最大堆，那么按层次顺序存放T的数组a[n]中的元素具有如下的性质
  - 若$2i + 1 < n$，则$a[i] \geq a[2i + 1]$
  - 若$2i + 2 < n$，则$a[i] \geq a[2i + 2]$
- 如果具有n个结点的完全二叉数T是一个最小堆，那么按层次顺序存放T的数组a[n]中的元素具有如下的性质
  - 若$2i + 1 < n$，则$a[i] \leq a[2i + 1]$
  - 若$2i + 2 < n$，则$a[i] \leq a[2i + 2]$

最小堆与最大堆



最小堆



最大堆

最小堆的类定义

```
#define DefaultSize 10;
template <class Type> class MinHeap : public MinPQ <Type> {
    private:
        Type *heap;  // 存放最小堆元素的数组
        int currentSize;  // 最小堆当前元素个数
        int maxHeapSize;  // 最多允许元素个数
        void SiftDown(int start, int m);  // 自顶向下进行调整成为最小堆
        void SiftUp(int start);  // 自底向上进行调整成为最小堆
    public:
        MinHeap(int sz);  // 构造函数：建立空堆
        MinHeap(Type arr[ ], int n);  // 构造函数
        ~MinHeap( ) { delete [ ] heap; }
        bool Insert(const Type &x);  // 插入
        bool Remove(Type &x);  // 删除
        bool IsEmpty( ) const { return currentSize == 0; }
        bool IsFull( ) const { return currentSize == maxHeapSize; }
};
```

最小堆的建立

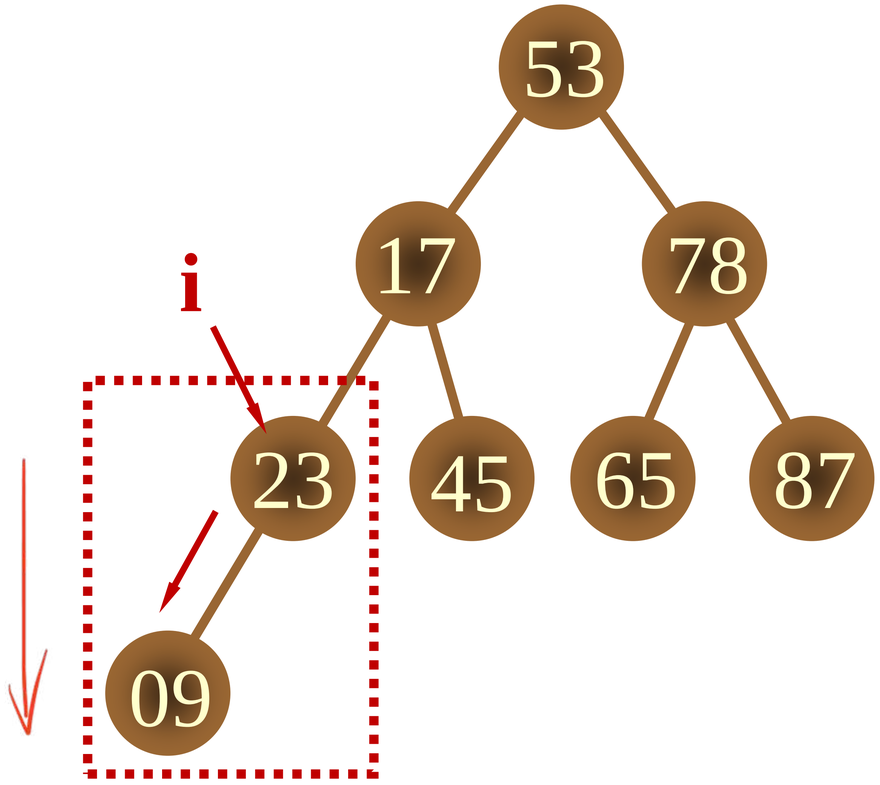
```
// 根据给定大小sz，建立堆对象
template <class Type> MinHeap <Type> ::
MinHeap(int sz) {
   // 确定堆的大小
   maxHeapSize = (DefaultSize < sz) ? sz : DefaultSize;
   heap = new Type [maxHeapSize];
   if (heap == NULL) {
      cerr<<“存储分配错！”<<endl;  exit(1);
   }
   currentSize = 0;
}
```

最小堆的建立

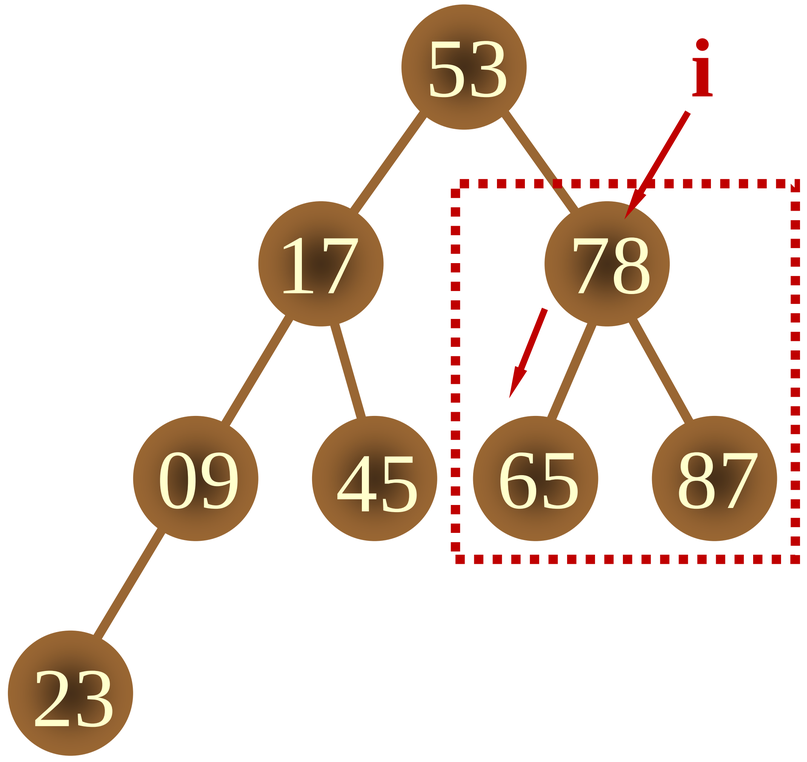
```
// 根据给定数组中的数据和大小，建立堆对象
template <class Type> MinHeap <Type> ::
MinHeap(Type arr[ ], int n) {
    maxHeapSize = DefaultSize < n ? n : DefaultSize;
    heap = new Type [maxHeapSize];
    if (heap == NULL) { cerr<<“存储分配错！ ”<<endl;  exit(1); }
    for (int i = 0; i < n; i++)  heap[i] = arr[i];  // 数组传送
    currentSize = n;  // 当前堆大小
    int currentPos = (currentSize-2)/2;  // 找最初调整位置：最后的非叶结点
    while (currentPos >= 0) {  // 从下到上逐步扩大，形成堆
       // 从currentPos开始，到CurrentSize止调整
       SiftDown(currentPos, currentSize - 1);
       currentPos--;
    }
}
```

currentSize+1−1 / 2

将用数组存放的任意数据调整成堆

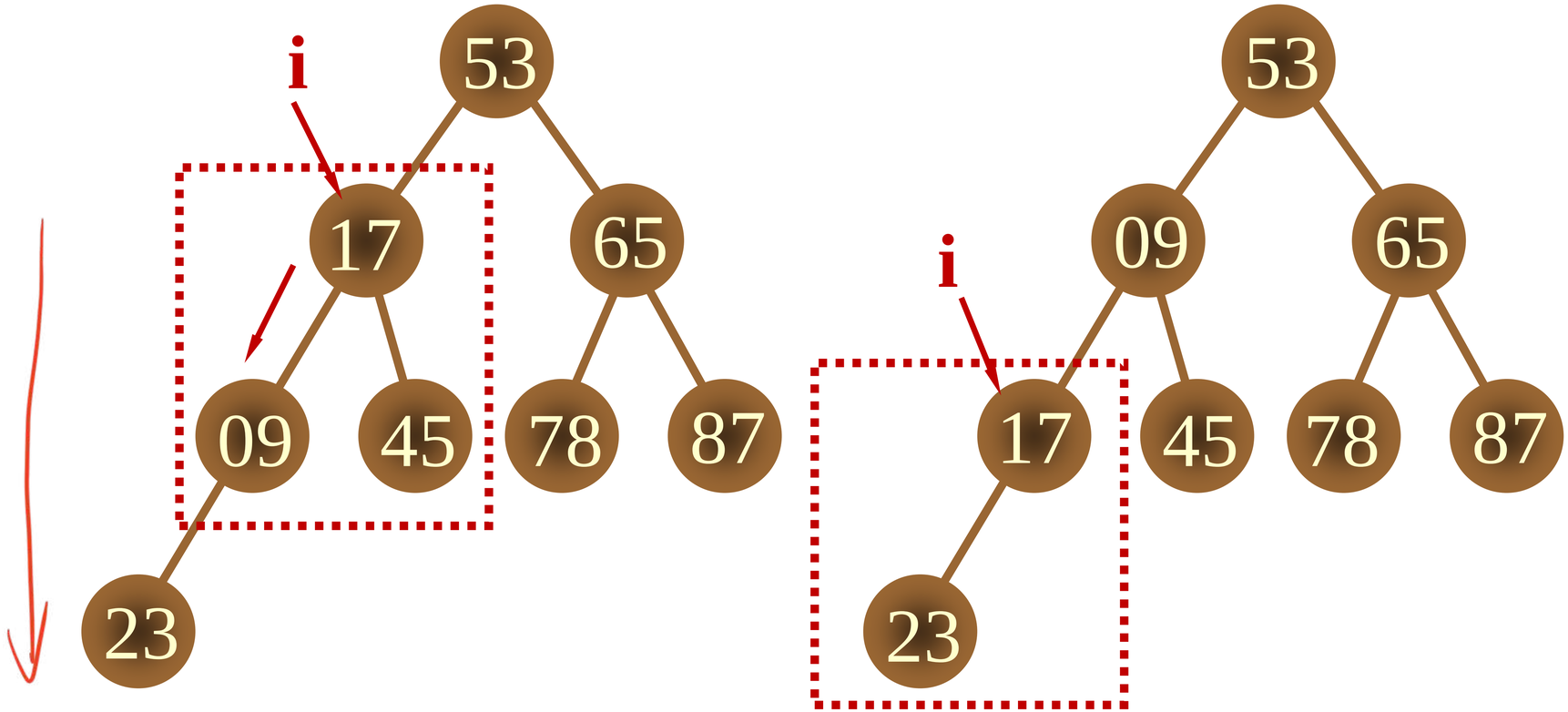


currentPos=i=3



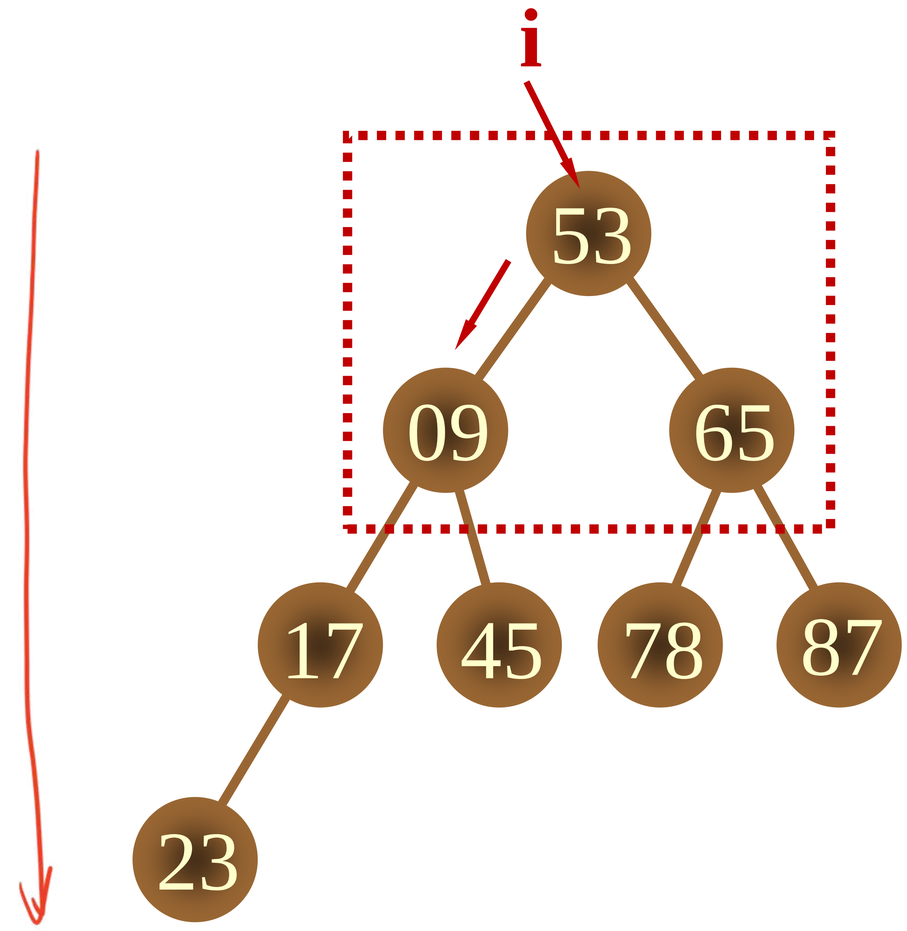
currentPos=i=2

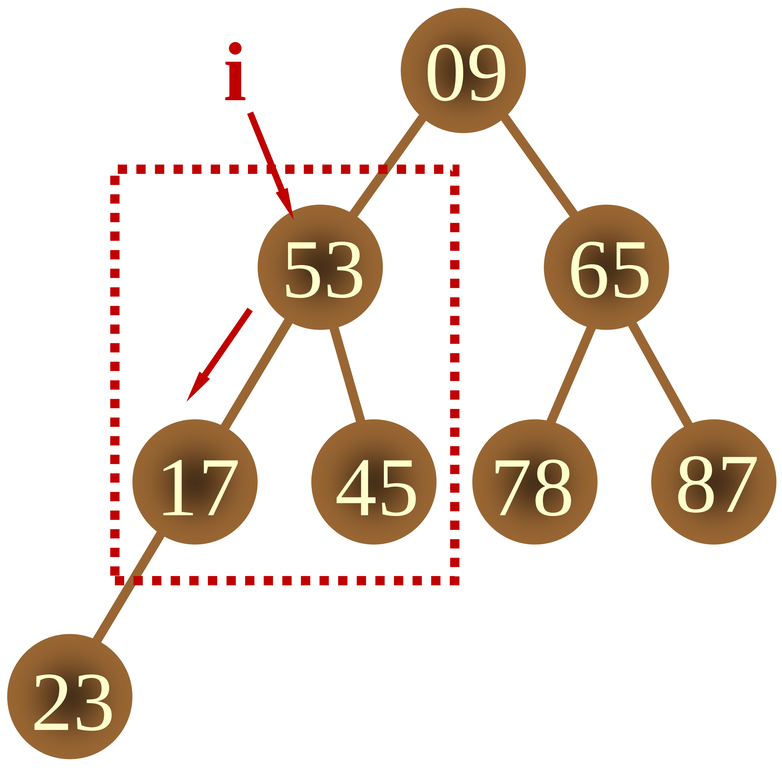
将用数组存放的任意数据调整成堆



currentPos=i=1

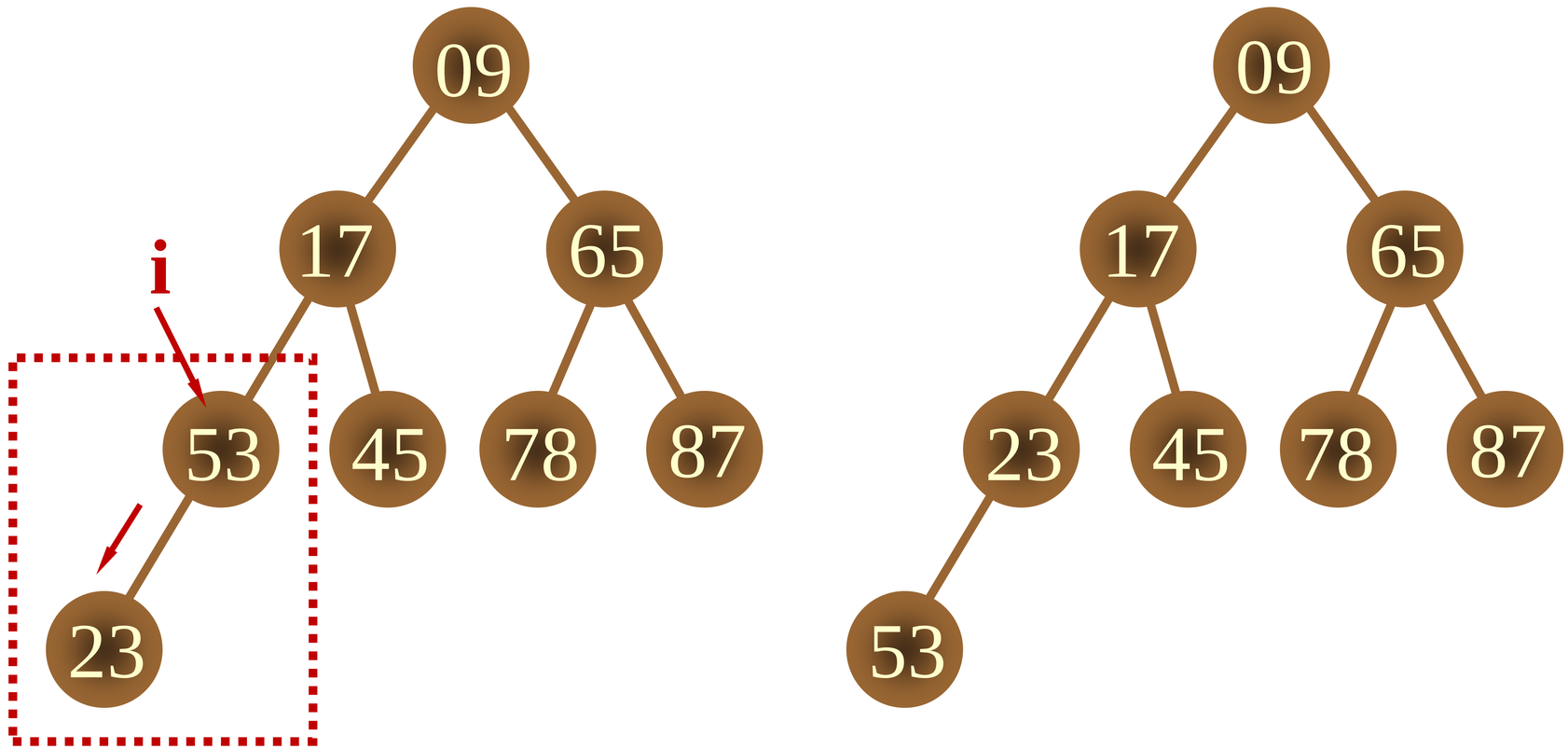
将用数组存放的任意数据调整成堆





currentPos=i=0

将用数组存放的任意数据调整成堆



最小堆的向下调整算法

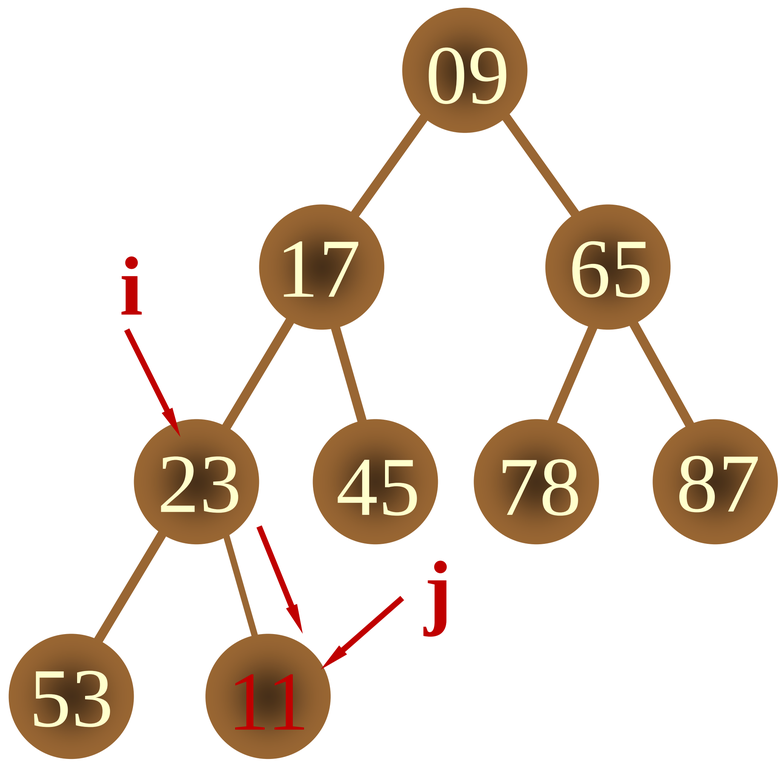
```
template <class Type> void MinHeap <Type> ::
SiftDown(int start, int m) {
    int i = start, j = 2 * i + 1;  // j是i的左子女
    Type temp = heap[i];
    while (j <= m) {
        if (j < m && heap[j] > heap[j+1])
            j++;  // 两子女中选小者
        if (temp <= heap[j])  break;
        else {
            heap[i] = heap[j];  // 下面的上浮
            i = j;  j = 2*j+1;  // 向下滑动
        }
    }
    heap[i] = temp;
}
```

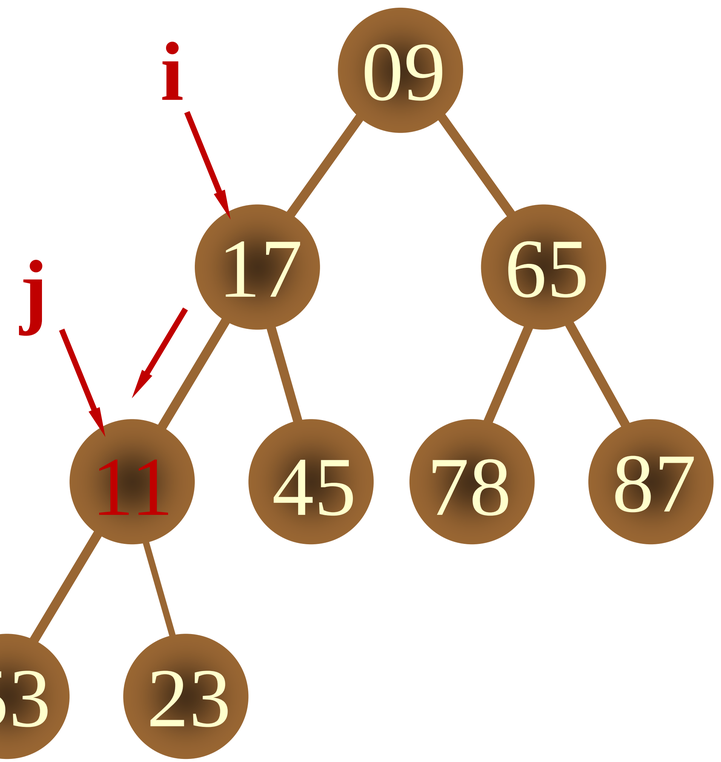
最小堆的插入

```
// 在最小堆中插入新元素x
template <class Type> bool MinHeap <Type> ::
Insert(const Type &x) {
    if (currentSize == maxHeapSize) {
        cerr<<“堆已满”<<endl;  return false;
    }
    heap[currentSize] = x;  // 插在数组末尾
    SiftUp(currentSize);  // 向上调整为最小堆
    currentSize++;  // 堆元素增一
    return true;
}
```

将currentSize位置元素上滑

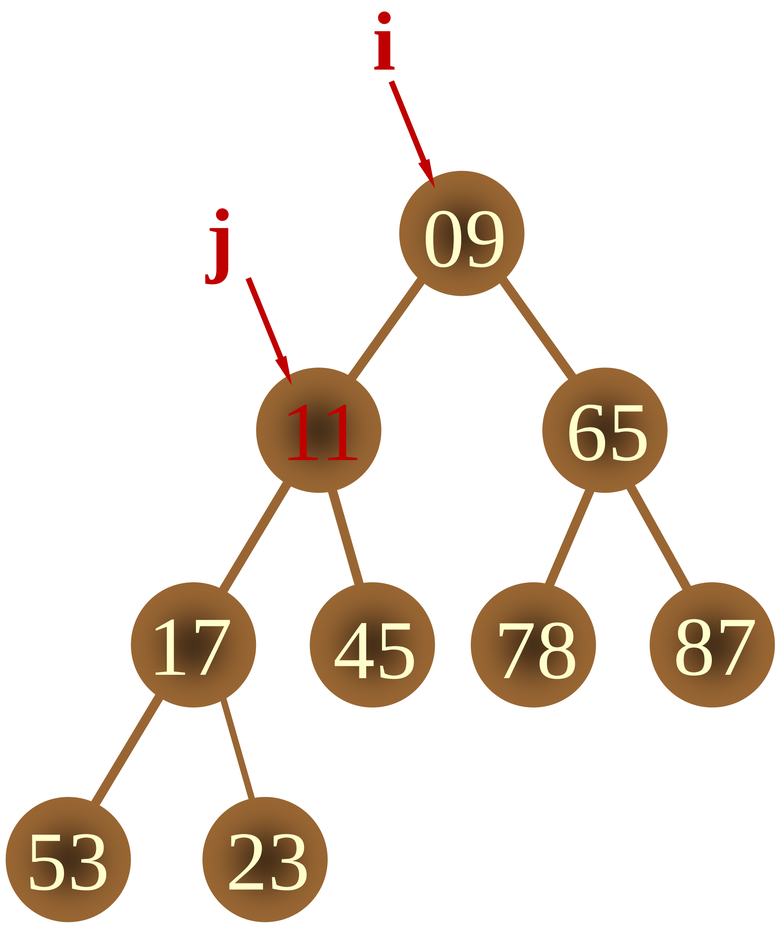
插入元素后调整成堆

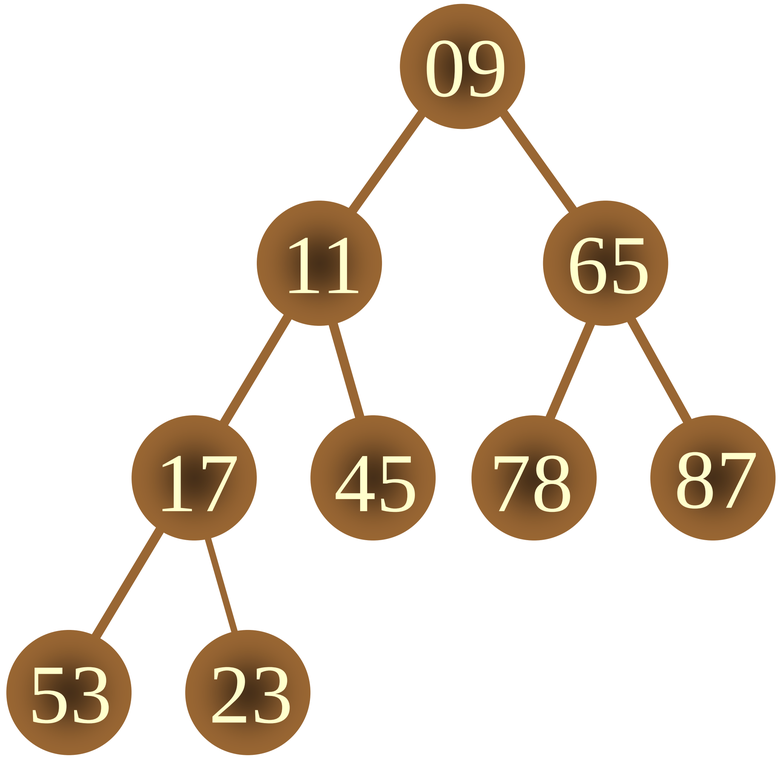




在堆中插入新元素11

插入元素后调整成堆





最小堆的向上调整算法

```
// 从start开始，向上直到0，调整堆
template <class Type> void MinHeap <Type> ::
SiftUp(int start) {
    int j = start, i = (j - 1) / 2;  // i是j的父结点
    Type temp = heap[j];
    while (j > 0) {
        if (heap[i] <= temp)  break;  // 父结点值小，不调整
        else { heap[j] = heap[i];  j = i;  i =(i - 1) / 2; }
    }
    heap[j] = temp;    // 回填
}
```

最小堆的删除

- 从最小堆删除具有最小值元素的操作，是将最小堆的堆顶元素(即完全二叉树的顺序表示的第0号元素)删除
- 在将该元素取走后，一般以堆的最后一个结点填补取走的堆顶元素，并将堆的实际元素个数减1
- 用最后一个元素取代堆顶元素将破坏堆，需要从堆顶向下进行调整

```
template <class Type> bool MinHeap <Type> ::
Remove(Type &x) {
    if (CurrentSize == 0) { cout<<“堆已空”<<endl;  return false; }
    x = heap[0];  // 删除最小元素
    heap[0] = heap[currentSize-1];  // 用最后元素填补到根结点
    currentSize--;
    SiftDown(0, currentSize-1);  // 向下调整
    return true;
}
```

一次即可

堆排序

- **堆排序是借助堆结构的排序**
  - **以最大堆为例，如果a[0], a[1], …, a[n-1]表示一个最大堆，那么a[0]是最大的，将a[0]与a[n-1]对调，把最大值调到a[n-1]**
  - **对a[0], a[1], …, a[n-2]进行向下调整，使之成为一个最大堆，调整后，a[0]是a[0], a[1], …, a[n-2]中最大的，也是全部结点中值第二大的，再将a[0]与a[n-2]对调，把值第二大的结点调到a[n-2]**
- **如此重复进行，直到调整范围内只含一个结点a[0]为止，此时，a[0]是全部结点中值最小的**
- **这样处理之后，a[0], a[1], …, a[n-1]已按从小到大的次序排了序**

```
void heap_sort(int a[ ], int n) {
    int i;  int t;
    for (i =(n - 2) / 2; i >= 0; i--)  SiftDown(a, i, n);  // 调整为堆
    for (i = n-1; i > 0; i--) {  // 进行堆排序
        t = a[0];  a[0] = a[i];  a[i] = t;  SiftDown(a, 0, i);
    }
}
```

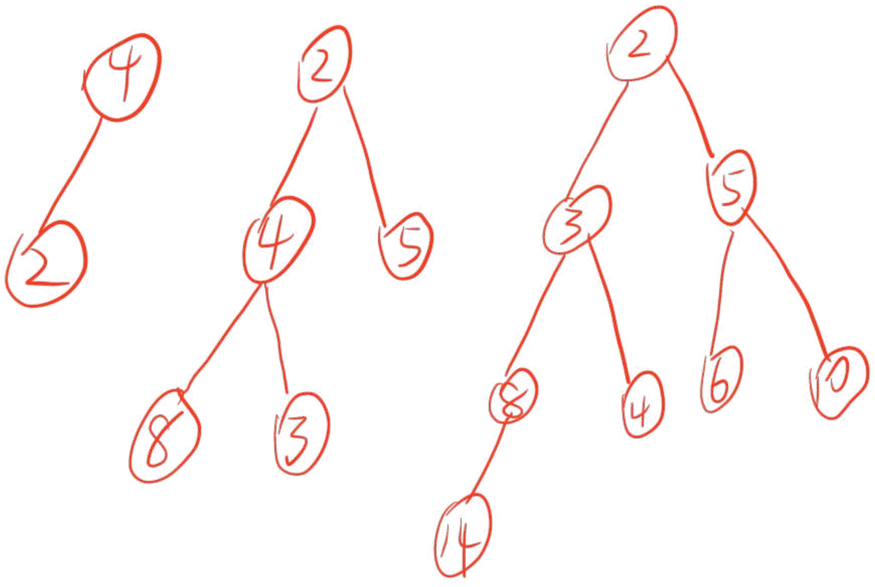
堆排序的时间复杂度

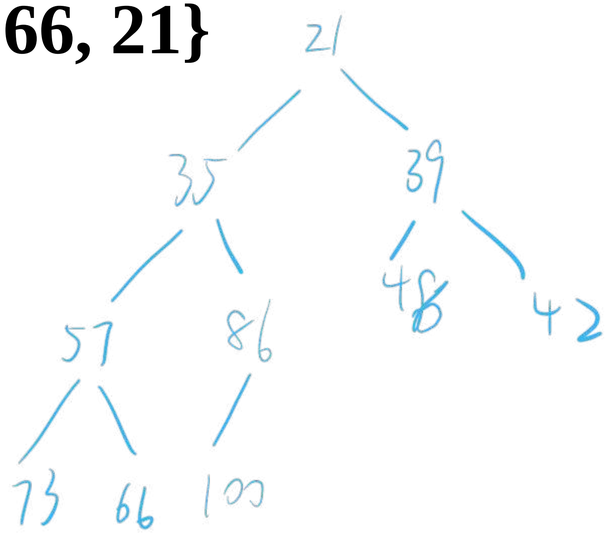
- 在SiftDown(a, i, n)的执行过程中，循环执行次数与表示堆的二叉树的层数有关，其执行次数不会超过树的层数
- 因为具有n个结点的堆的层数为$\lfloor \log_2 n \rfloor + 1$，故调用一次SiftDown(a, i, n)的执行时间不会超过$O(\log_2 n)$
- heap_sort(a, n)的执行时间：第一个循环最多执行n/2次，故执行时间为$O(n\log_2 n)$；而第二个循环的执行次数为(n - 1)，故执行时间为$O(n\log_2 n)$。因此，整个heap_sort(a, n)的执行时间为$O(n\log_2 n)$

练习2

- **写出向最小堆中加入数据4, 2, 5, 8, 3, 6, 10, 14时，每加入一个数据后堆的变化**

- **判断以下序列是否是最小堆？如果不是，调整为最小堆**
  - **{100, 86, 48, 73, 35, 39, 42, 57, 66, 21}**





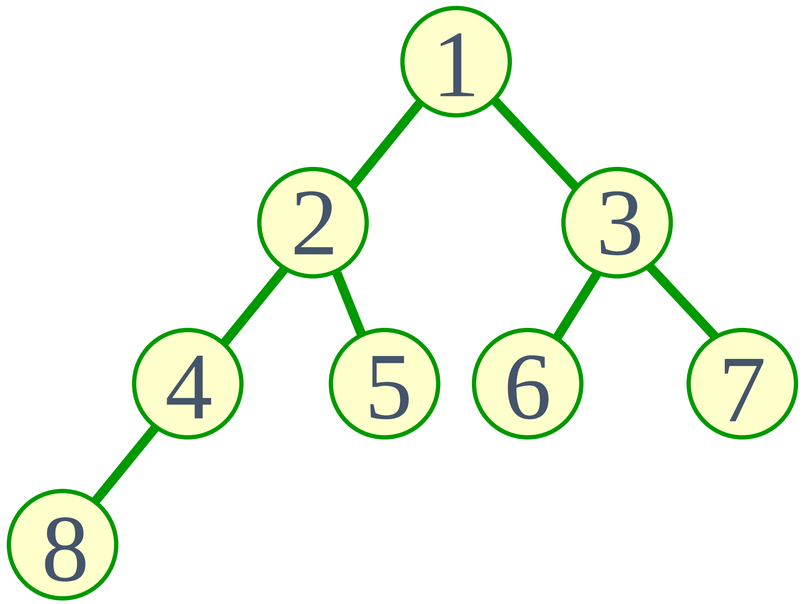
06 树

- 二叉树的计数
- 线索化二叉树
- 堆
- **霍夫曼树**

路径长度(Path Length)

- 两结点间的路径长度是连接两结点的路径上的边数
- 树的外部路径长度(EPL)是各叶结点(外结点)到根结点的路径长度之和
- 树的内部路径长度(IPL)是各非叶结点(内结点)到根结点的路径长度之和
- 树的路径长度(PL)是各结点到根结点的路径长度之和，即PL = EPL + IPL

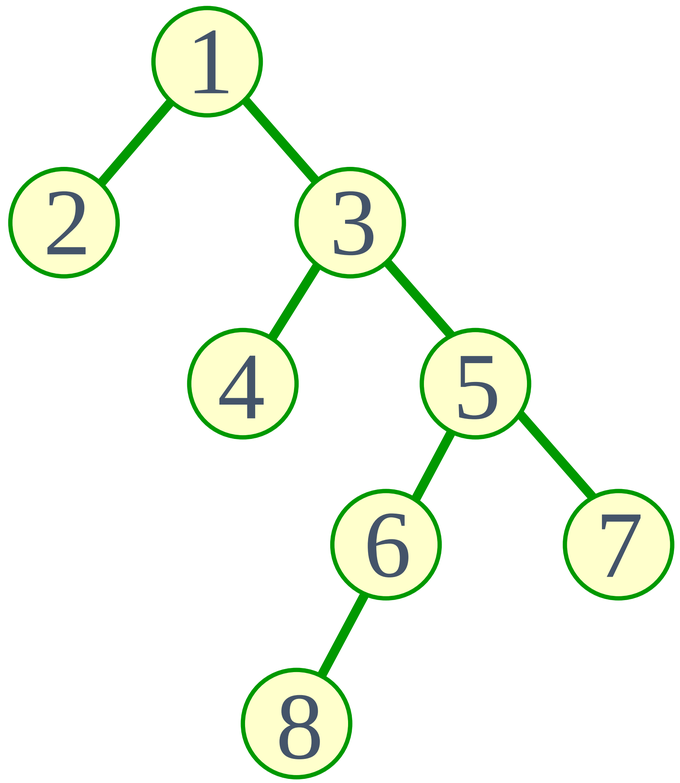
路径长度



树的路径长度

PL=0+1*2+2*4+3*1=13

完全二叉树



树的路径长度

PL=0+1*2+2*2+3*2+4*1=16

一般二叉树

路径长度

- n个结点的二叉树的路径长度不小于下述数列前n项的和

$$PL=\sum_{i=0}^{n-1}\lfloor \log_2(i+1)\rfloor$$

$$=0+1+1+2+2+2+2+3+3+\cdots$$

1×2^1　　2×2^2　　3×2^3

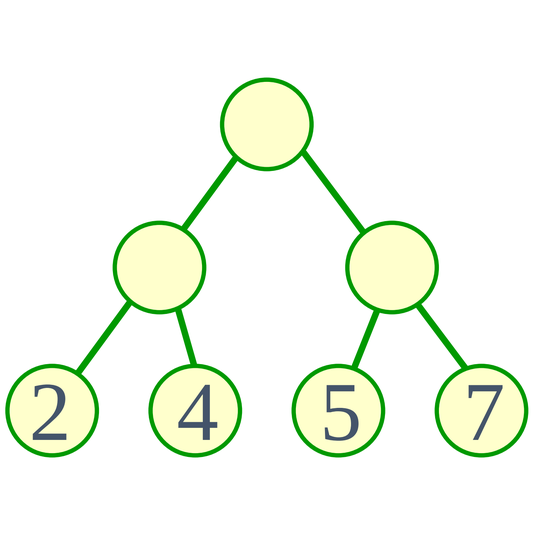
- 其路径长度最小者为 $PL=\sum_{i=0}^{n-1}\lfloor \log_2(i+1)\rfloor$

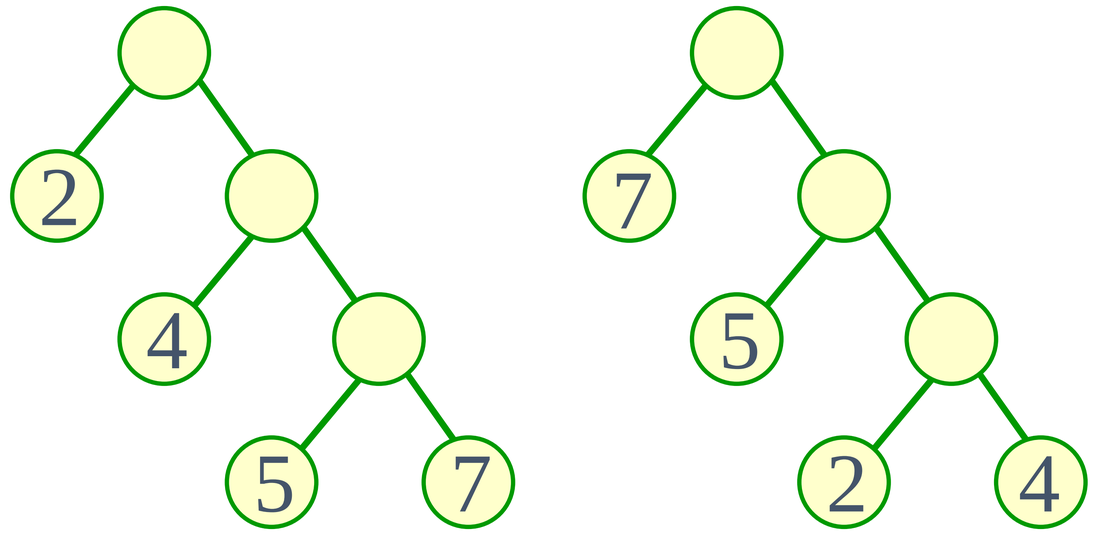
带权路径长度(Weighted Path Length)

- 给定一个有n个权值的集合$\{w_0, w_1, \ldots, w_{n-1}\}$，其中$w_i \geq 0$ $(0 \leq i \leq n-1)$。若T是一棵有n个叶结点的二叉树，而且将权值$w_0, w_1, \ldots, w_{n-1}$分别赋给T的n个叶结点，则称T是权值为$w_0, w_1, \ldots, w_{n-1}$的扩充二叉树
- 具有n个叶结点的扩充二叉树的带权路径长度为各叶结点所带的权值(w_i)与该结点到根的路径长度(l_i)的乘积的和

$$WPL = \sum_{i=0}^{n-1} w_i * l_i$$

带权路径长度





WPL=2*2+4*2+
5*2+7*2=36

WPL=2*1+4*2
+5*3+7*3=46

WPL=7*1+5*2
+2*3+4*3=35

带权路径长度达到最小

带权路径最小的扩充二叉树不一定是完全二叉树

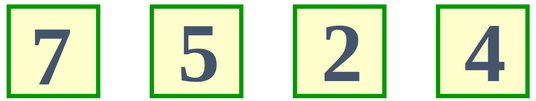
霍夫曼树(Huffman Tree)

- 带权路径长度达到最小的扩充二叉树即为**霍夫曼树**
- 在霍夫曼树中，权值越大的结点离根越近

- **霍夫曼算法**：构造权值为$w_0, w_1, \ldots, w_{n-1}$的霍夫曼树
- 由给定的n个权值$w_0, w_1, \ldots, w_{n-1}$，构造具有n棵扩充二叉树的森林$F=\{T_0, T_1, \ldots, T_{n-1}\}$，其中每棵扩充二叉树$T_i$只有一个带权值$w_i$的根结点，其左、右子树均为空
- 重复以下步骤，直到F中仅剩下一棵树为止
  - 在F中选取两棵根结点的权值最小的扩充二叉树，作为左、右子树构造一棵新的二叉树，置新的二叉树的根结点的权值为其左、右子树上根结点的权值之和
  - 在F中删去这两棵二叉树
  - 把新的二叉树加入F

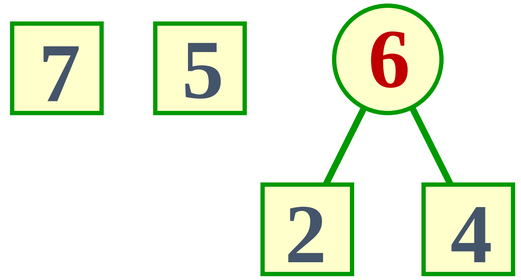
霍夫曼算法过程示例

F : {7} {5} {2} {4}



初始

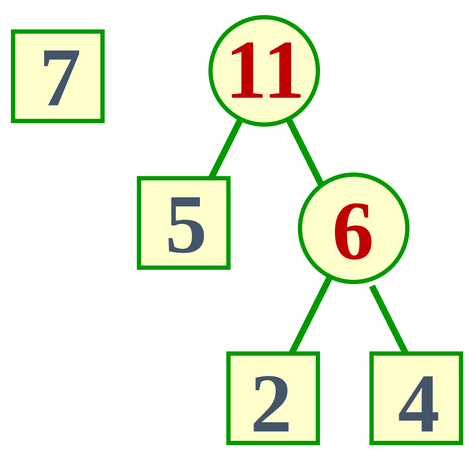
F : {7} {5} {6}



合并{2} {4}

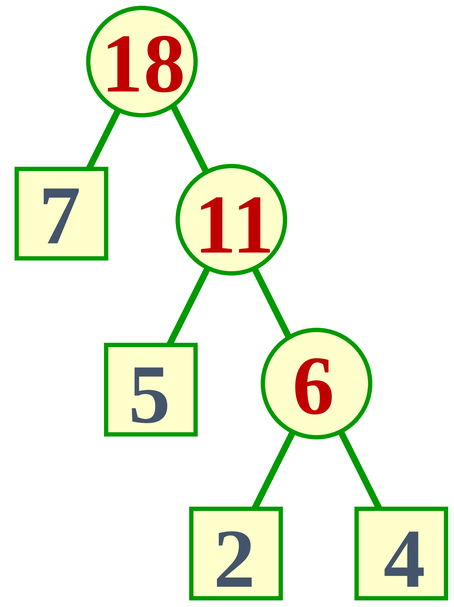
霍夫曼算法过程示例

F : {7} {11}



合并{5} {6}

F : {18}



合并{7} {11}

霍夫曼树的类定义

```
const  int DefaultSize=20;
template <class Type> class HuffmanTree;
template <class Type> class HuffmanNode {
    friend class HuffmanTree;
    private:
        Type data;
        HuffmanNode <Type> *leftChild, *rightChild, *parent;
    public:
        HuffmanNode(Type elem,
                            HuffmanNode <Type> *left=NULL,
                            HuffmanNode <Type> *right=NULL,
                            HuffmanNode <Type> *pr=NULL) :
          data(elem), leftChild(left), rightChild(right), parent(pr) { }
        ……
};
```

霍夫曼树的类定义

```
template <class Type> class HuffmanTree {
    public:
        HuffmanTree(Type w[ ], int n);
        ~HuffmanTree( ) { DeleteTree(root); }
    protected:
        HuffmanNode <Type> *root;
        void DeleteTree(HuffmanNode <Type> *t);
        void MergeTree(HuffmanNode <Type> &ht1,
                       HuffmanNode <Type> &ht2,
                       HuffmanNode <Type> *&parent);
};
```

霍夫曼算法

```
template <class Type> HuffmanTree <Type> ::
HuffmanTree(Type w[ ],  int n) {
    MinHeap < HuffmanNode <Type> > hp;
    HuffmanNode <Type> * parent, first, second, work;
    for (int i = 0; i < n; i++) {
        work = new HuffmanNode;
        work.data = w[i];  work.leftChild = NULL;
        work.rightChild = NULL;  work.parent = NULL;
        hp.Insert(work);
    }
    for (int i = 0; i < n-1; i++) {
        hp.RemoveMin(first);  hp.RemoveMin(second);
        MergeTree(first, second, parent);  hp.Insert(*parent);
    }
    root = parent;
}
```

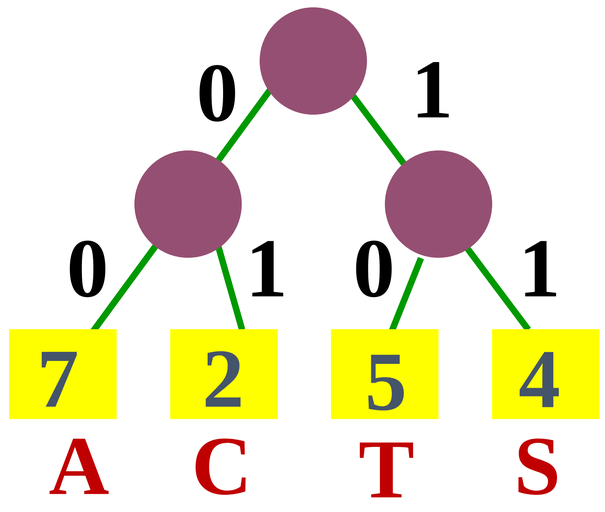
霍夫曼算法

```
template <class Type> void HuffmanTree <Type> ::
MergeTree(HuffmanNode  <Type> &bt1,
                  HuffmanNode <Type> &bt2,
                  HuffmanNode <Type> *&parent) {
    parent = new HuffmanNode;
    parent->leftChild = &bt1;
    parent->rightChild = &bt2;
    parent->data = bt1.root->data + bt2.root->data;
    bt1.root->parent = bt2.root->parent = parent;
}
```

霍夫曼树的应用：霍夫曼编码

- 霍夫曼编码的主要用途是实现数据压缩

- 设给出一段报文：CAST CAST SAT AT A TASA
- 字符集合是{C, A, S, T}，各个字符出现的频度(次数)是W＝{2, 7, 4, 5}
- 若给每个字符以等长编码，如A : 00, T : 10, C : 01, S : 11，则总编码长度为(2+7+4+5)*2=36

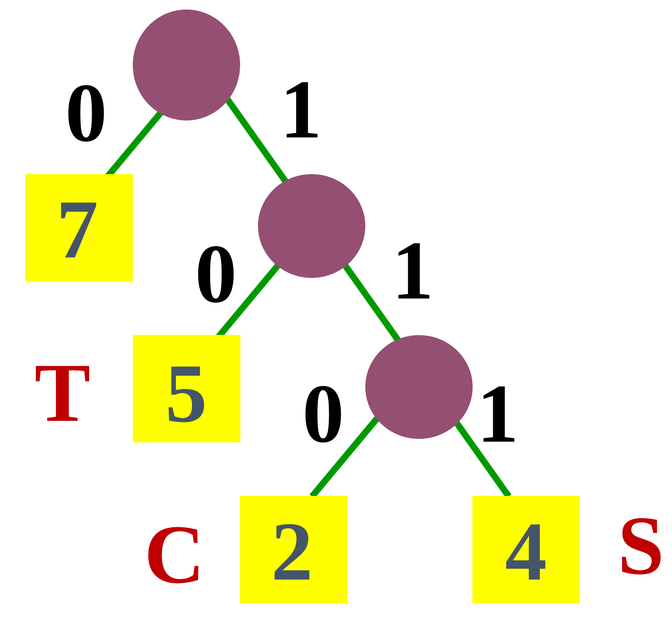


霍夫曼树的应用：霍夫曼编码

- 若按各个字符出现的概率不同而给予不等长编码，可以减少总编码长度
- 各字符出现概率为{2/18, 7/18, 4/18, 5/18 }，化整为{2, 7, 4, 5}，以它们为各叶结点上的权值建立霍夫曼树，左分支赋0，右分支赋1，得到霍夫曼编码(变长编码)
  - 即A : 0, T : 10, C : 110, S : 111
  - 其总编码长度为7*1+5*2+(2+4)*3 = 35

树状结构使得每一个字符的编码都不是另一个的前缀

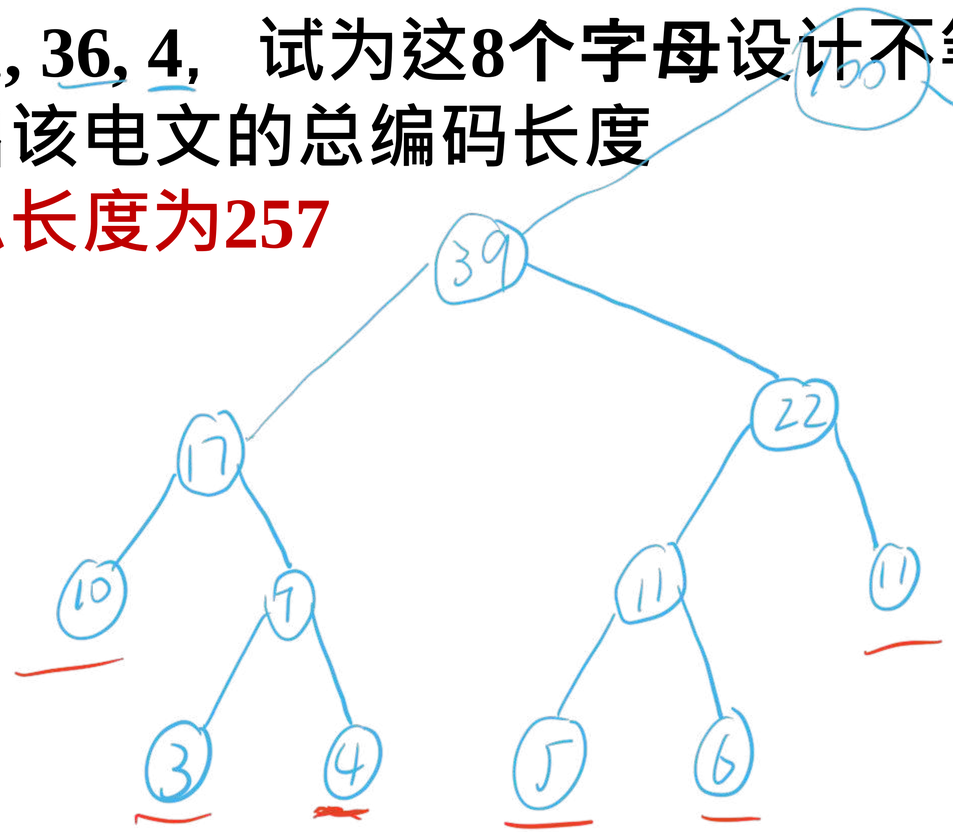
  - 霍夫曼编码是一种无前缀编码
  - 解码时不会混淆



霍夫曼编码树

练习3

- 假定用于通信的电文仅由8个字母c1, c2, c3, c4, c5, c6, c7, c8组成，各字母在电文中出现的频率分别为5, 25, 3, 6, 10, 11, 36, 4，试为这8个字母设计不等长霍夫曼编码，并给出该电文的总编码长度
- 总长度为257



$(25+36)\times2+(10+11)\times3+(3+4+5+6)\times4$

$=122+63+72$

$=257<100\times3$

本章小结

- 二叉树的计数
- 线索化二叉树
- 堆
- 霍夫曼树

作业

- 试编写一个算法，把一个新结点l作为结点s的左子女插入到中序线索化二叉树中，s原来的左子女变成l的左子女

- 判断以下序列是否是最小堆？如果不是，调整为最小堆
  - (1) {12, 70, 33, 65, 24, 56, 48, 92, 86, 33}
  - (2) {103, 97, 56, 38, 66, 23, 42, 12, 30, 52, 06, 20}
  - (3) {05, 56, 20, 23, 40, 38, 29, 61, 35, 76, 28, 100}

- 给定权值集合{15, 03, 14, 02, 06, 09, 16, 17}，构造相应的霍夫曼树，并计算它的带权路径长度